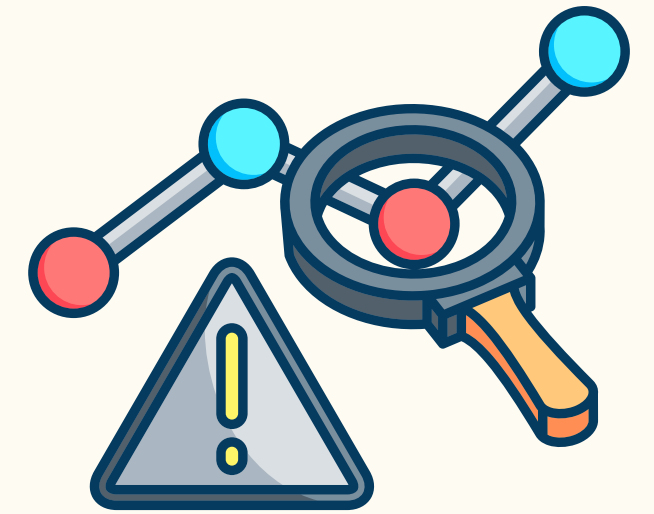




ANOMALY DETECTION IN INDUSTRIAL CONTROL SYSTEMS

STUDY PROJECT

GROUP 6: ALEKSA & SWETHA

February 05, 2025

WHAT IS AN INDUSTRIAL CONTROL SYSTEM (ICS)?

- An Industrial Control System (ICS) is a cyber-physical system that monitors and controls critical infrastructures.
- Integrates IT (Information Technology) & OT (Operational Technology).
- Ensures real-time monitoring & automation.
- Highly vulnerable to cyber-physical threats.

Source: <https://www.alltialloys.com/blog-posts/permalloy-the-high-performance-alloy>



LAYERS - INDUSTRIAL CONTROL SYSTEM

3 layers:

1 Field Layer (Sensors & Actuators)

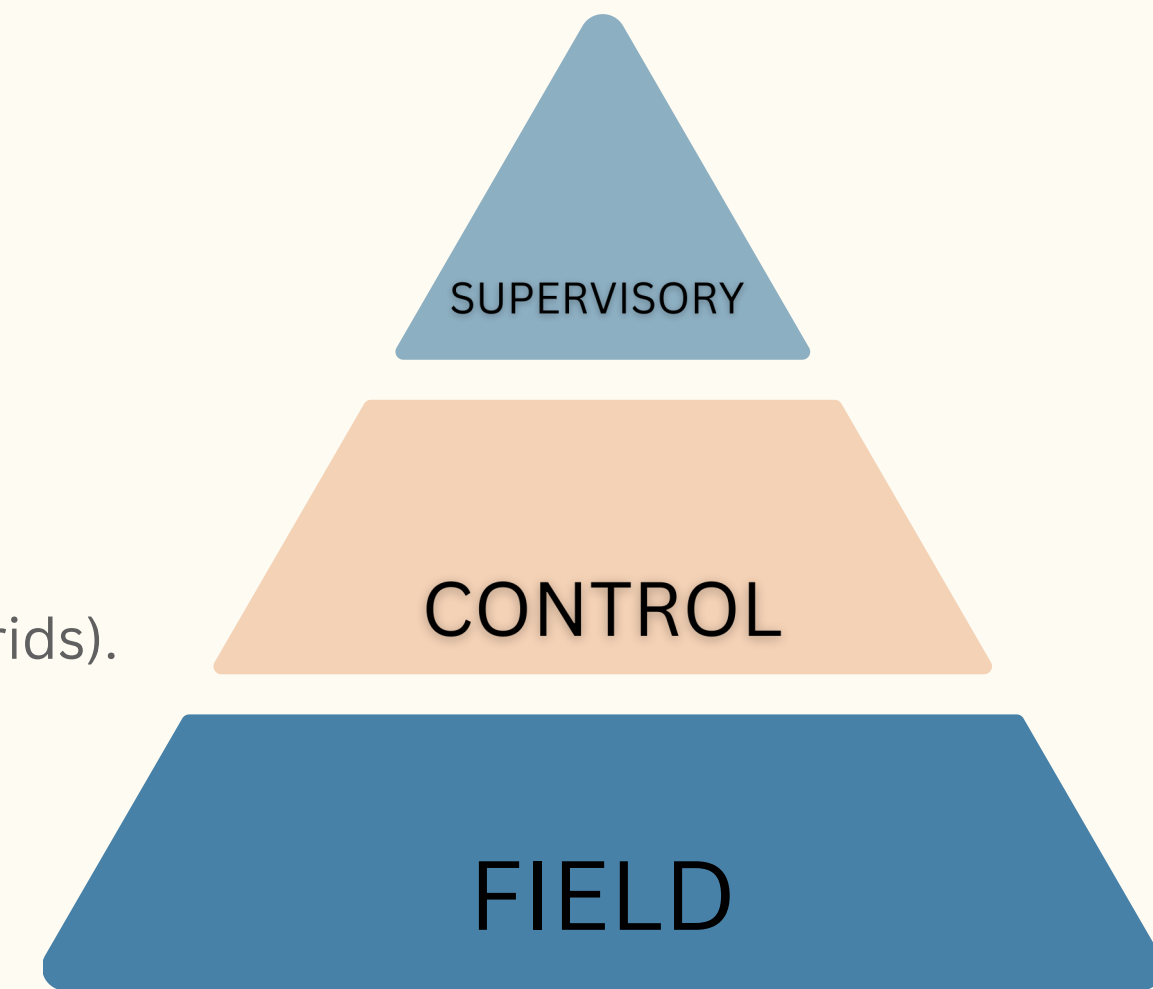
- Sensors (e.g., temperature, pressure) collect physical readings.
- Actuators (e.g., motors, valves) control physical processes.
- Communicates with controllers using industrial protocols (e.g., Modbus).

2 Control Layer (PLCs & RTUs)

- PLCs (Programmable Logic Controllers) process sensor data & make decisions.
- RTUs (Remote Terminal Units) manage remote operations (e.g., pipelines, power grids).
- Sends control signals to actuators based on predefined logic.

3 Supervisory Layer (SCADA & HMI)

- SCADA (Supervisory Control & Data Acquisition) monitors & controls industrial processes.
- HMI (Human-Machine Interface) provides a visual dashboard for operators.
- Collects & stores data for analysis, alarms, & remote management.



PROJECT GOAL

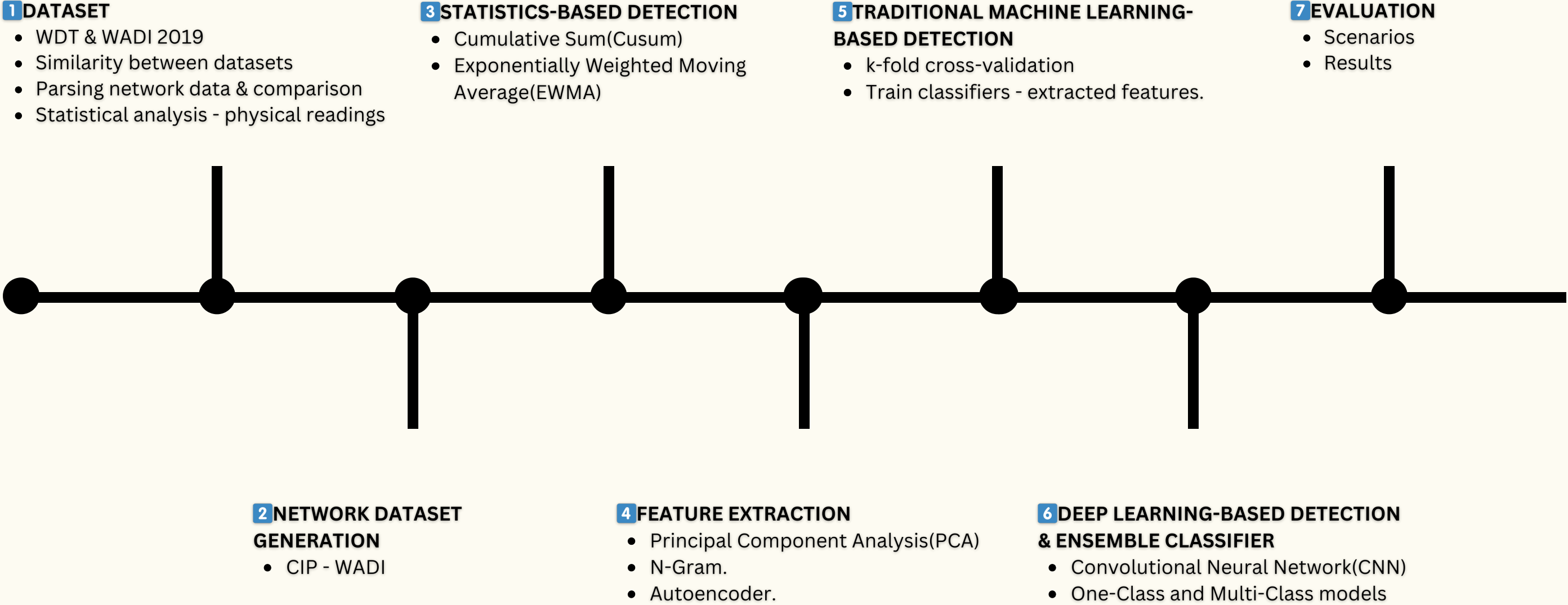


- ◆ Develop and evaluate advanced anomaly detection methods for Industrial Control Systems (ICS) using statistical, traditional machine learning, and deep learning approaches.

Why is This Important?

- ⚡ ICS Security is Critical: Power grids, water distribution, and gas pipelines rely on ICS for safe and efficient operation.
- 🌐 Increased Connectivity = Higher Risk: The integration of Operational Technology (OT) and IT introduces new cyber-physical attack surfaces.
- 🔍 Limitations of Traditional IDS: Existing Intrusion Detection Systems (IDS) struggle to differentiate between normal variations and real threats, leading to false alarms or undetected attacks.

METHODOLOGY



DATASETS



	WDT	WADI 2019
Source	Acquired from a water distribution testbed	Designed by iTrust, SUTD for ICS security research
Infrastructure	8 tanks, 22 solenoid valves, 6 pumps, 8 pressure sensors, 4 flow sensors, and 8 manual valves	123 sensors and actuators
Testbed Components	- Physical partition (real hardware) - Simulated partition (virtual connection)	Fully operational water distribution system
Duration	4 sessions (~2 hours total)	16 days total 14 days normal 2 days attack scenarios
Size	Physical dataset ~ 4.4 MB Network dataset ~10.4 GB	Physical dataset ~575 MB

SIMILARITY BETWEEN DATASETS

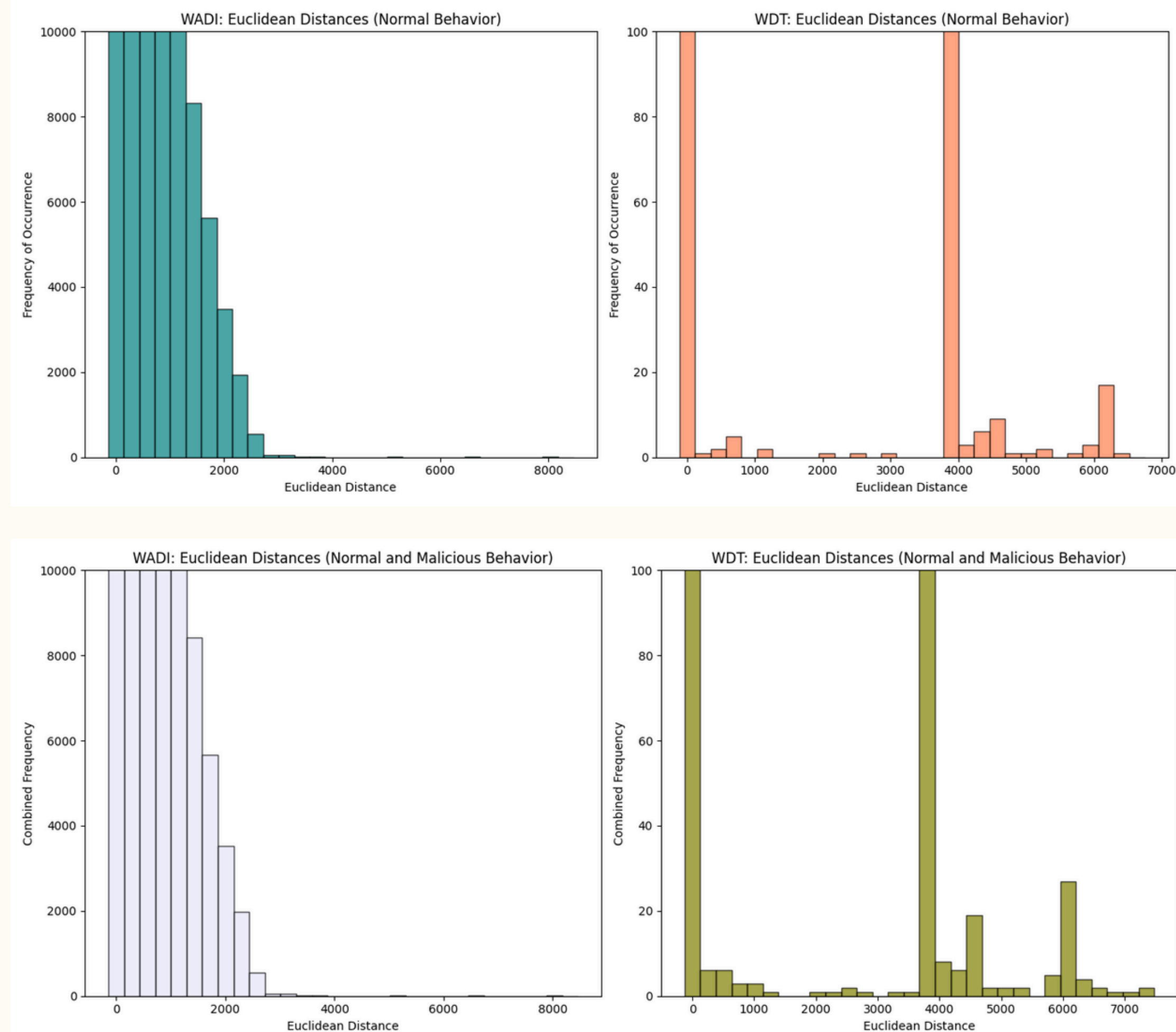
- ◆ Analyze WDT and WADI 2019 datasets to compare physical readings.
- ◆ Compute and visualize Euclidean distance between time steps to detect variations.
- ◆ Separate computation for:
 - Normal behavior only
 - Normal + malicious behavior.

WHAT DOES THIS ACHIEVE?

- Compare datasets(WDT and WADI 2019) to see how similar they are.
- Understanding dataset differences ensures that machine learning models generalize well.

SIMILARITY BETWEEN DATASETS

- This histogram visualizes the contrast between normal and anomalous data, which is important for feature selection and model performance.
- Large Euclidean distances in the combined behavior plot indicate attacks or anomalies, whereas smaller distances in the normal behavior plot suggest stable, expected behavior.



PARSING NETWORK PACKETS & DATA COMPARISON

- ◆ Use Scapy to process raw pcap files.
- ◆ Convert extracted data into a human-readable format.

Key Steps:

- ◆ Parse Modbus TCP packets to retrieve sensor/actuator readings.
- ◆ Store parsed data in a structured table (rows: time periods, columns: sensors/actuators).
- ◆ Compute MinMax statistics for parsed readings:
 - Calculate min/max values from original dataset.
 - Introduce error margin to define range.
 - Identify deviations in newly parsed data.

$$\begin{aligned}\text{minerr} &= \text{min} - (\text{max} - \text{min}) / 2 \\ \text{maxerr} &= \text{max} + (\text{max} - \text{min}) / 2\end{aligned}$$



STATISTICAL ANALYSIS - PHYSICAL READINGS

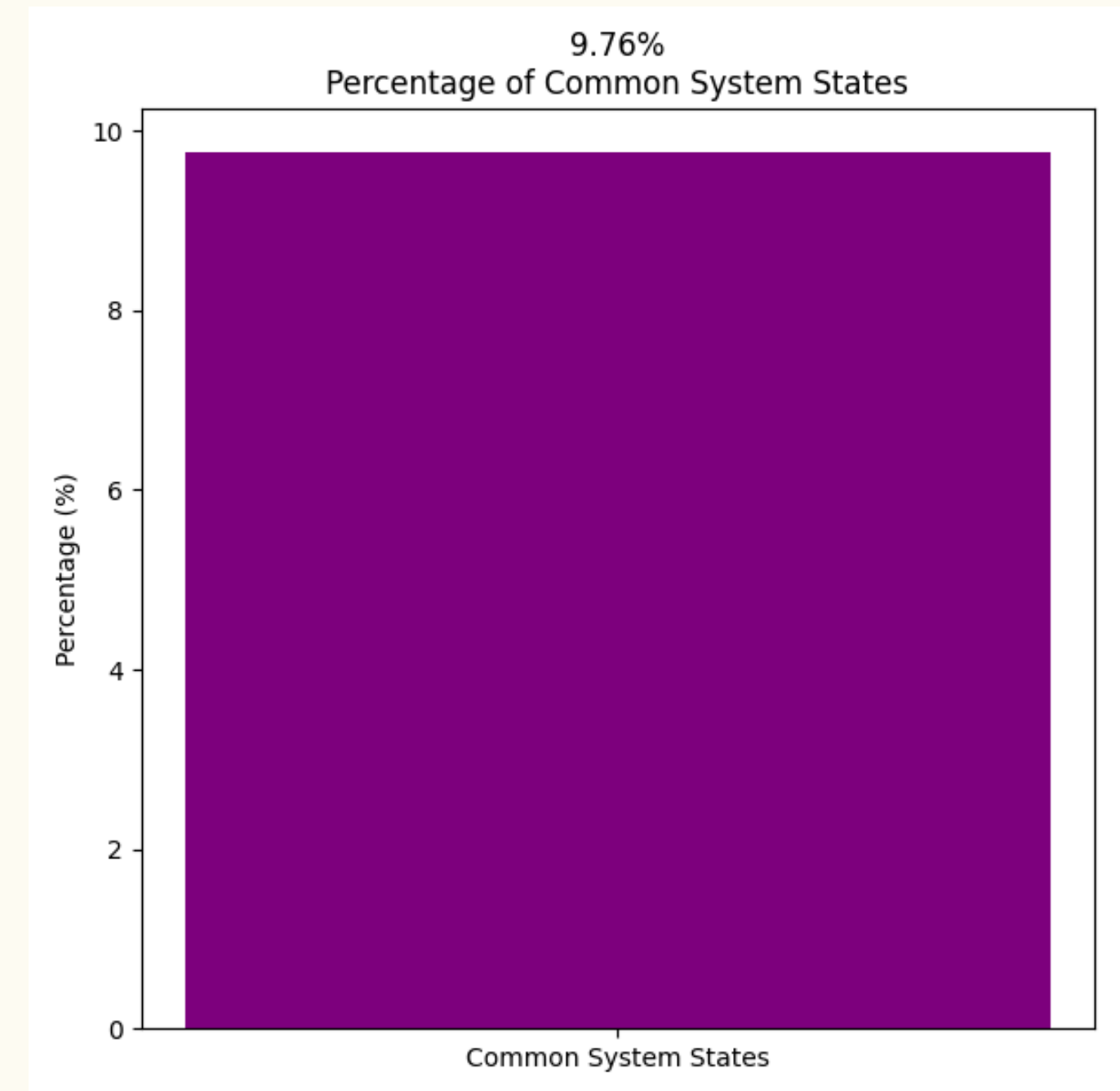
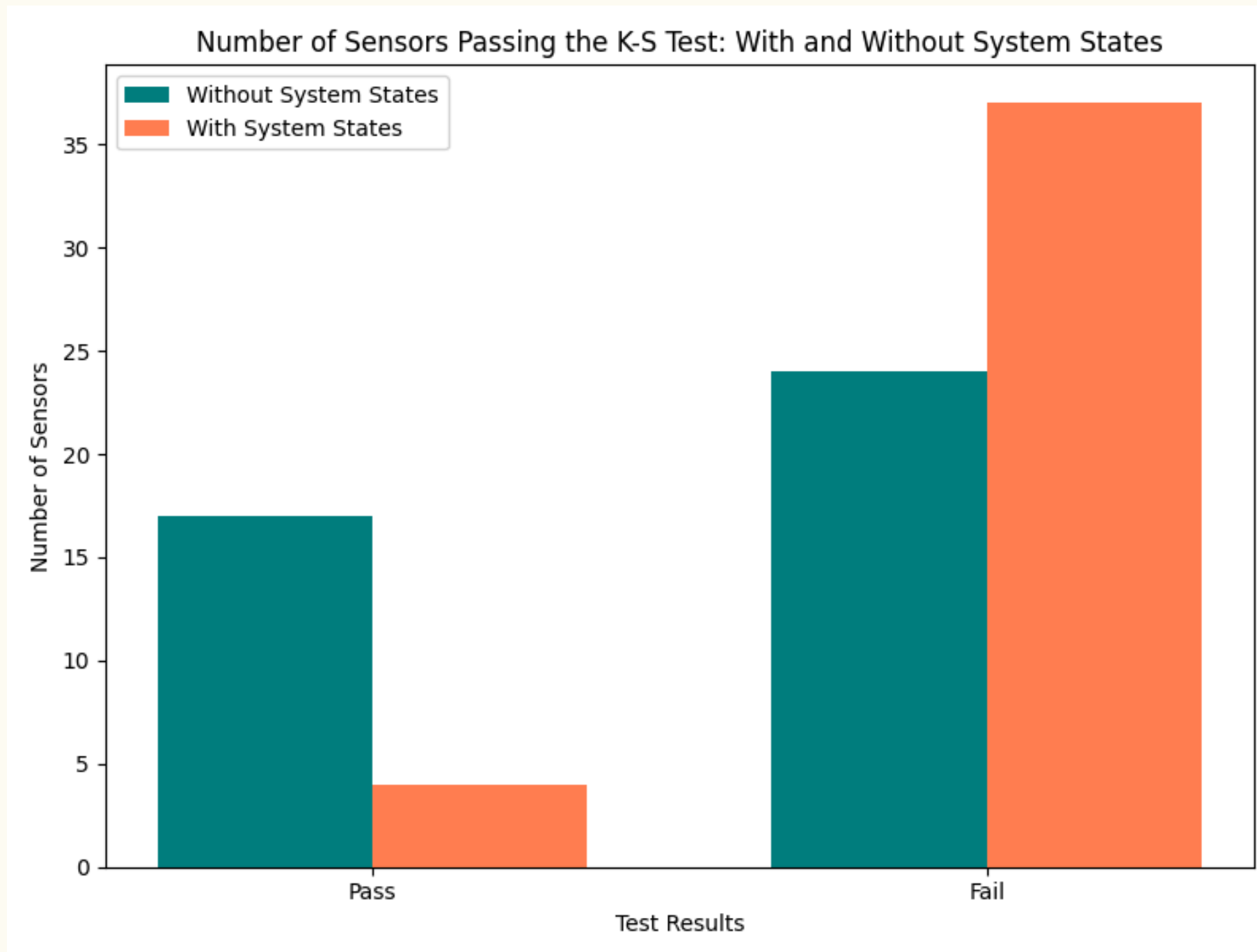
- ◆ Apply statistical techniques to detect stable patterns in normal sensor readings.
- ◆ Use the Kolmogorov-Smirnov (K-S) test to check if training and testing sets follow similar distributions.
- ◆ Identify common system states (sensor & actuator values occurring together).

WHY IS THIS IMPORTANT?

- ◆ Machine learning models work best when train and test data follow the same statistical distribution.
- ◆ If training data doesn't match real-world testing data, the model will fail to detect anomalies properly.
- ◆ Identifying system states helps in reducing false alarms by considering contextual dependencies.



STATISTICAL ANALYSIS - PHYSICAL READINGS



- It assesses the statistical similarity between training and testing data distributions.
- 9% common system state suggests that there is only a small overlap of system states between training and testing sets.

NETWORK DATASET GENERATION

- ◆ Convert sensor readings into network traffic using CIP(Common Industrial Protocol) in WADI dataset.
- ◆ Simulate realistic ICS network data for intrusion detection.

WHY GENERATE NETWORK DATASET?

- ✓ Needs Both Network & Sensor-Based Detection
-> comparison for future experiments
- ✓ Testing Intrusion Detection on Realistic Network Data

Function codes	Command	Protocol
0x4C	Read Data	CIP
0x4D	Write Data	CIP

STATISTICS-BASED DETECTION

FEATURE	CUSUM(Cumulative Sum)	EWMA(Exponentially Weighted Moving Average)
Formula	$S_0=0, S_{k+1} = \max(0, S_k + r_k - \beta)$ r_k : current measurement - previous measurement β : threshold	$S_0 = y_0, S_{k+1} = \alpha \cdot y_{k+1} + (1 - \alpha) \cdot S_k$ y_k : Current sensor reading. α : Weighting factor.
Deviation	Slow, Gradual anomalies	Sudden changes
Sensitivity	Small deviations over time	Short-term fluctuations
Computational Cost	Low (suitable for real-time)	Low (suitable for real-time)
Example	A slow modification of sensor readings (e.g., an attacker gradually increases temperature values to avoid detection).	A sudden sensor spike (e.g., a pressure sensor jumps from 20 to 80 instantly due to a cyber-attack manipulating Modbus packets).

- ◆ Optimization of parameters α and β - counting the amount of false alarms

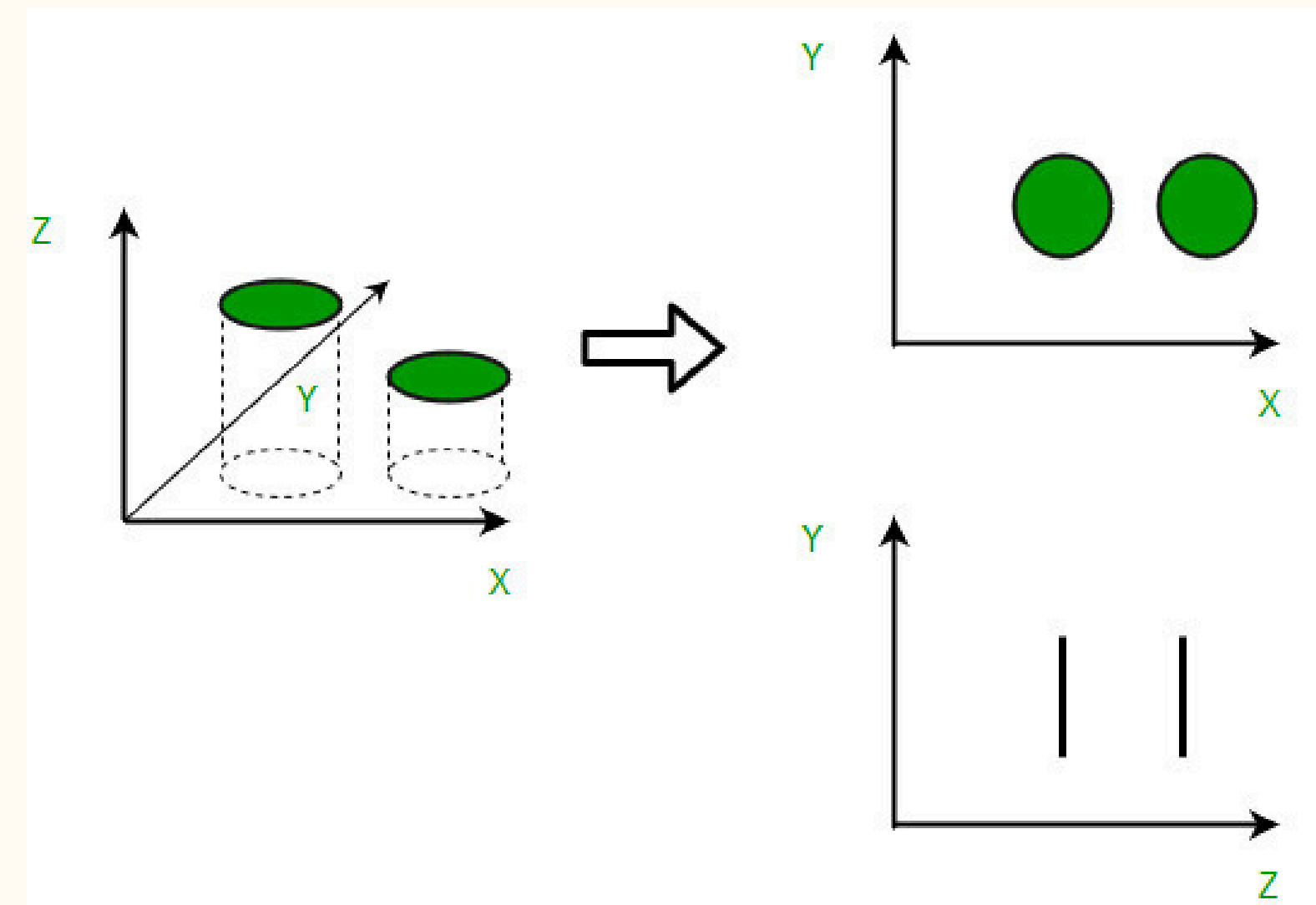
FEATURE EXTRACTION

1. FEATURE EXTRACTION - PCA (PRINCIPAL COMPONENT ANALYSIS)

- ◆ Reduce dataset dimensionality

WORKING?

- 1 Computes the covariance matrix to understand relationships between features.
- 2 Finds eigenvalues & eigenvectors to identify the most significant components.
- 3 Selects top principal components that explain most of the variance.



Source: <https://beyondknowledgeinnovation.ai/unsupervised-learning/dimensionality-reduction/>

FEATURE EXTRACTION

2. FEATURE GENERATION - AUTOENCODER

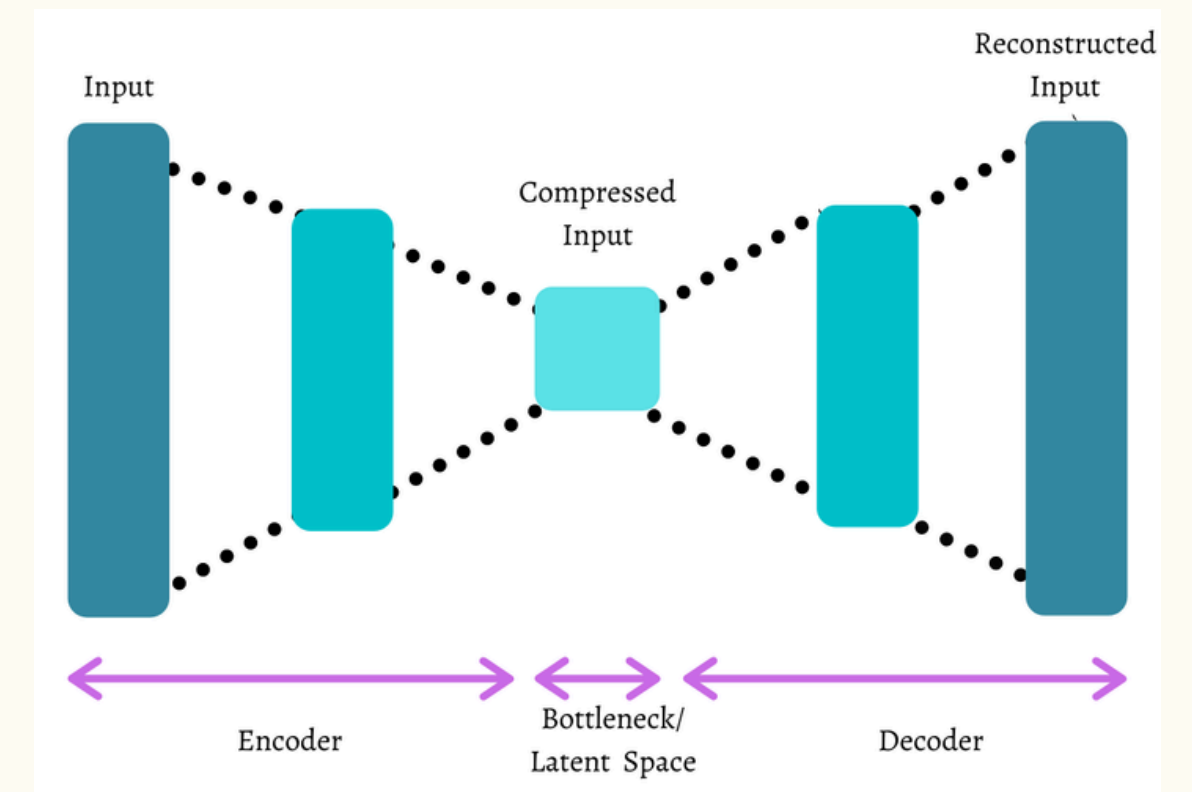
- A deep learning model that learns compressed representations of data.
- It consists of:
 - Encoder: Compresses input into a lower-dimensional representation.
 - Decoder: Reconstructs input from the compressed representation.

How Does It Work?

- 1 Train the autoencoder on data to learn important features.
- 2 Use the encoded layer as the extracted feature representation.
- 3 High reconstruction error = Potential anomaly detected.

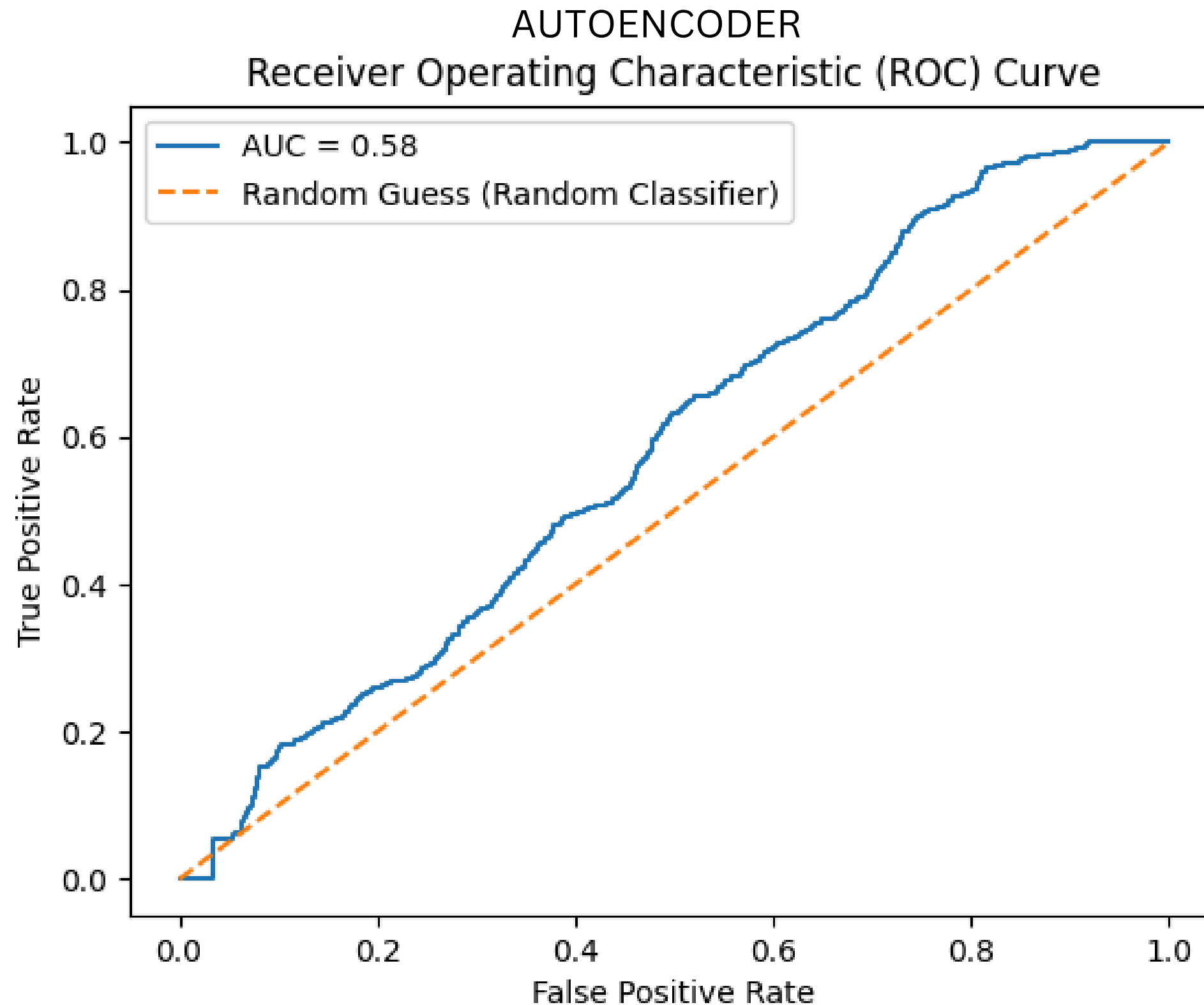
Why is Autoencoder Important?

- ✓ Learns complex feature relationships that traditional methods miss.
- ✓ Can be trained unsupervised (no attack labels required).
- ✓ Useful for detecting unseen anomalies in ICS data.



Source: <https://towardsdatascience.com/autoencoders-in-practice-dimensionality-reduction-and-image-denoising-ed9b9201e7e1>

FEATURE EXTRACTION



- A distribution plot showing reconstruction errors for normal and anomalous data.
- The curve shows that the autoencoder is identifying attacks without classifying too many normal instances as attacks

FEATURE EXTRACTION

3. FEATURE EXTRACTION - N-GRAMS

- An N-Gram is a sequence of N values.
- Feature sets: Generate N-grams
 - Without smoothing: From raw data sequences.
 - With smoothing: Apply a smoothing technique

HOW DOES IT WORK?

- 1 Convert time-series sensor readings into overlapping sequences.
- 2 Extract patterns of normal behavior using historical data.
- 3 Identify anomalies when new patterns do not match expected sequences.

WHY IS N-GRAM IMPORTANT?

- ✓ Works well for detecting recurring attack patterns.
- ✓ Can be applied to both sensor & network data.

TRADITIONAL ML-BASED DETECTION

K- FOLD CROSS-VALIDATION

- A technique used to split the dataset into multiple training and testing sets to improve model evaluation.

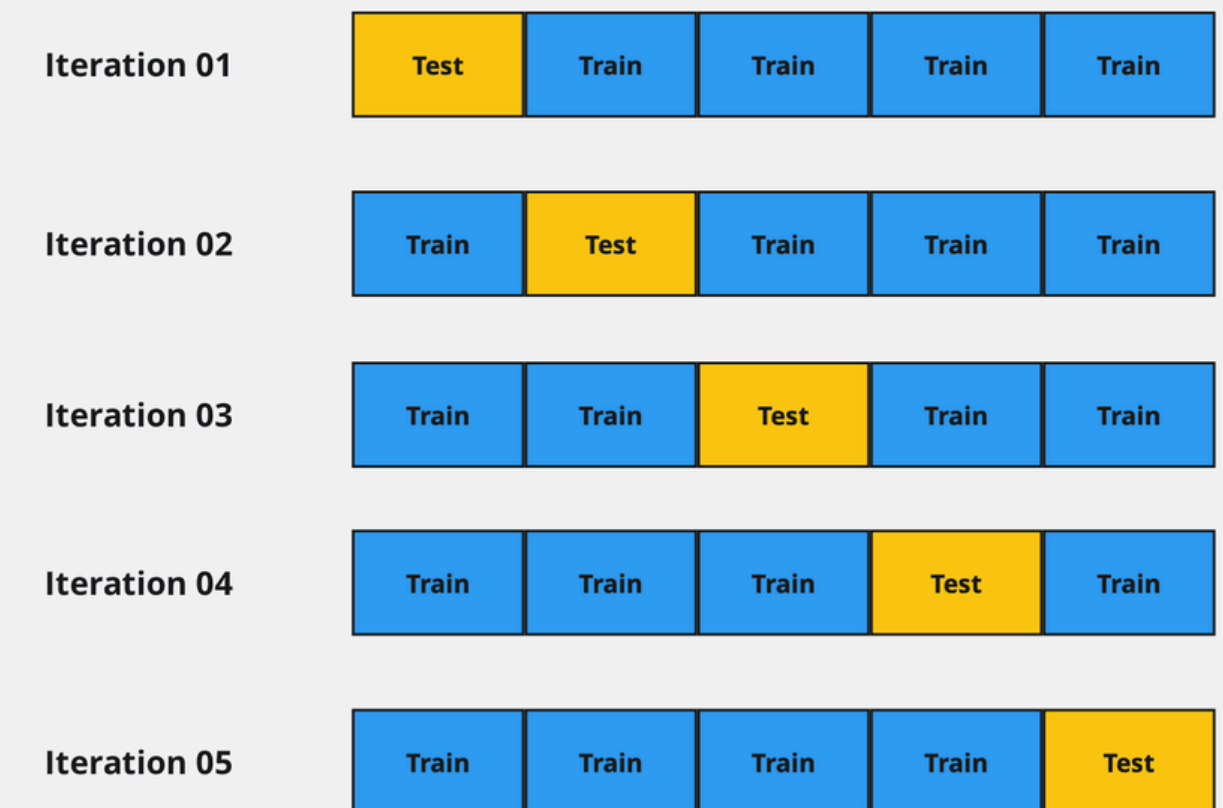
HOW DOES IT WORK?

- 1 Divide the dataset into K equal parts (**Folds**).
- 2 Train the model on K-1 folds and test on the remaining fold.
- 3 Repeat K times, using a different fold as the test set each time.
- 4 Compute the average performance across all K runs.

WHY K-FOLD?

- ✓ Reduces bias by training on different parts of the dataset.
- ✓ Prevents overfitting to a single dataset split.

K-Fold Cross Validation



Source: <https://dataaspirant.com/cross-validation/>

SCENARIOS

SCENARIO

1

Train Data: Only normal sensor readings (no attacks).

Test Data: Contains normal readings + all attack types (each attack type equally represented).

2

Train Data: Normal data + $n-1$ attack types.

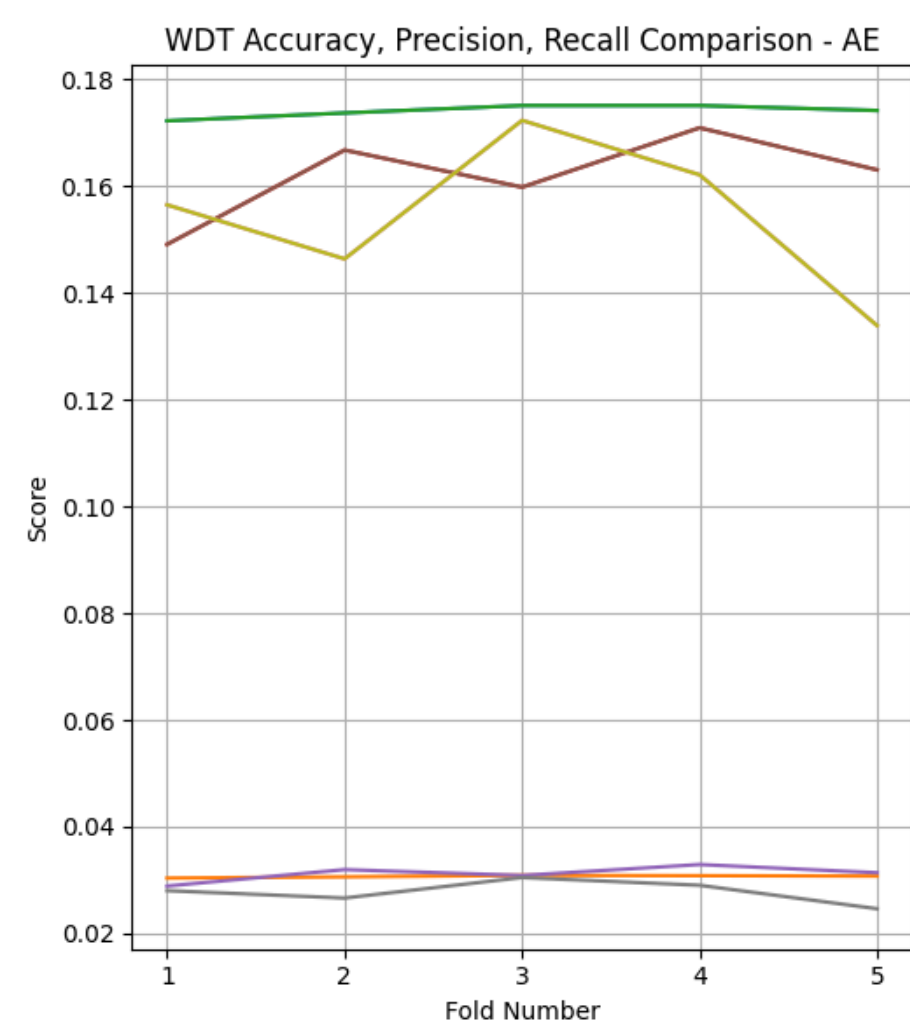
Test Data: Normal data + all attack types.

3

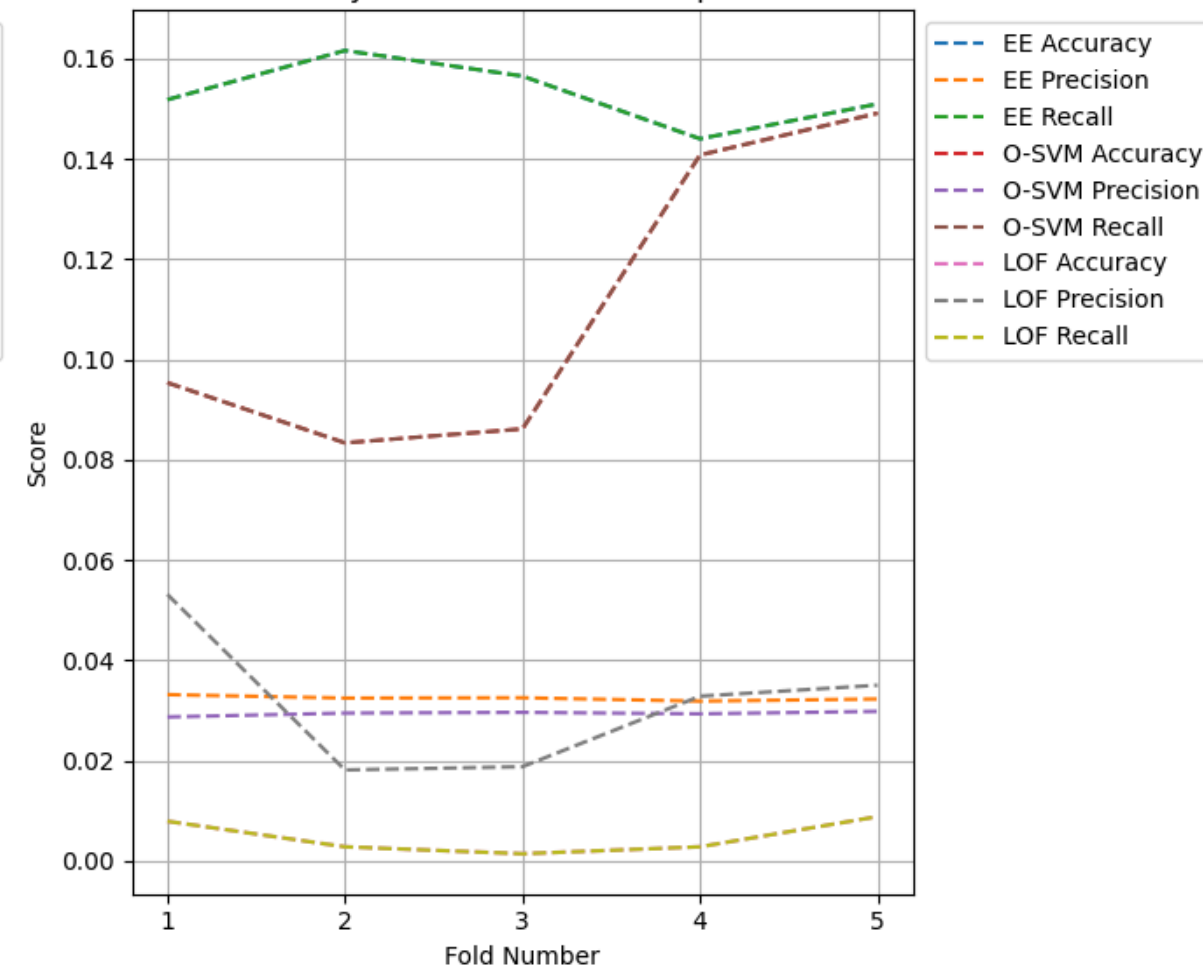
Train Data: Normal data + only one attack type.

Test Data: Normal data + all attack types

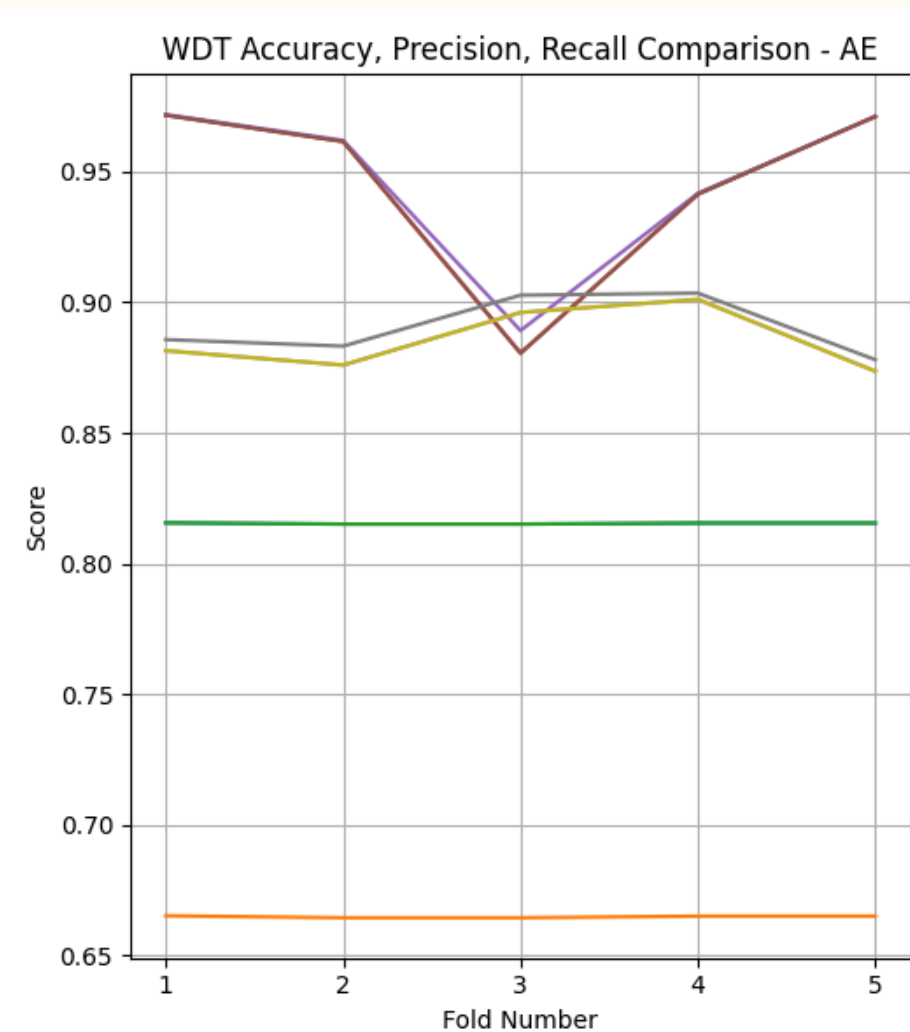
RESULTS



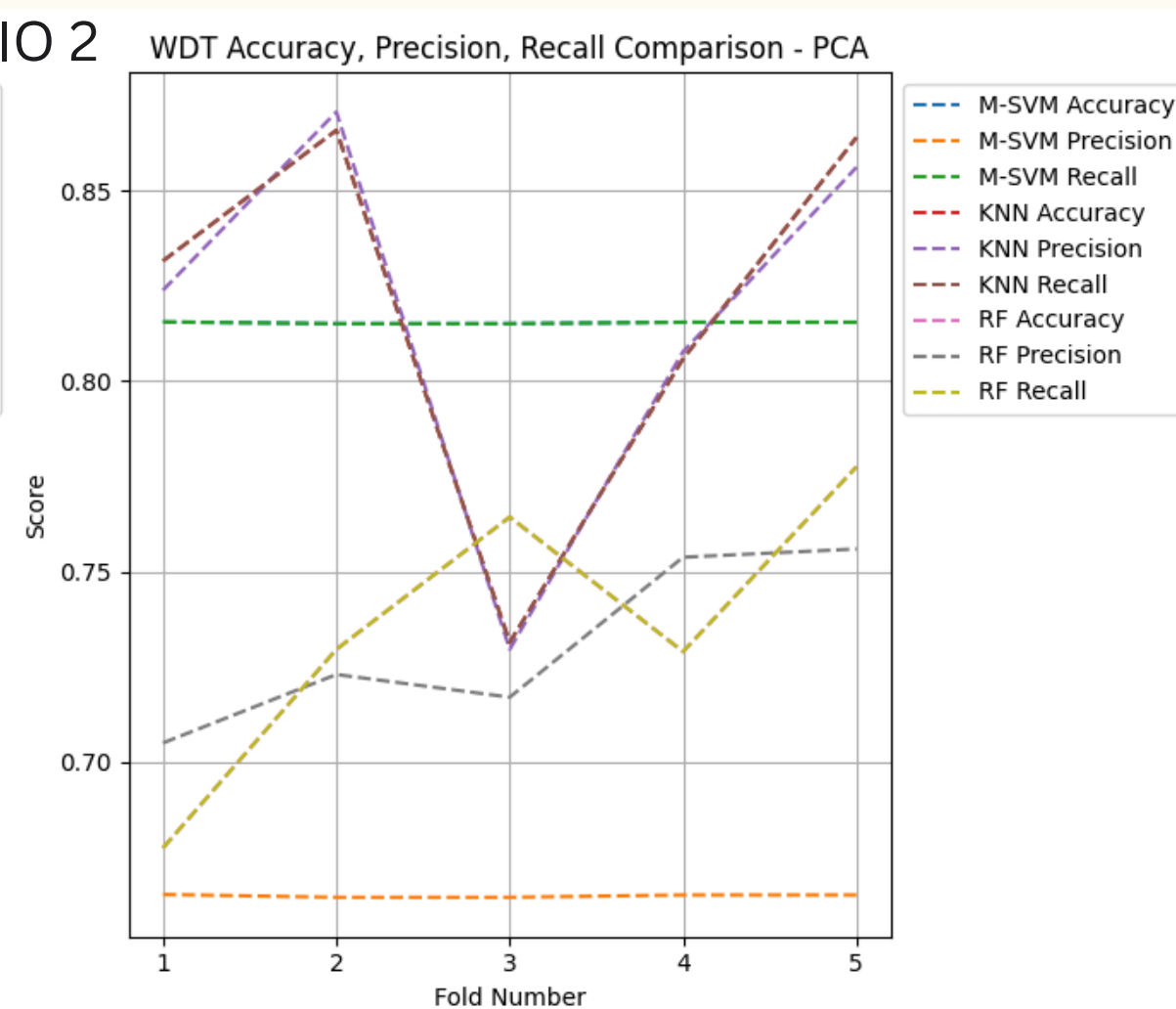
SCENARIO 1 WDT Accuracy, Precision, Recall Comparison - PCA



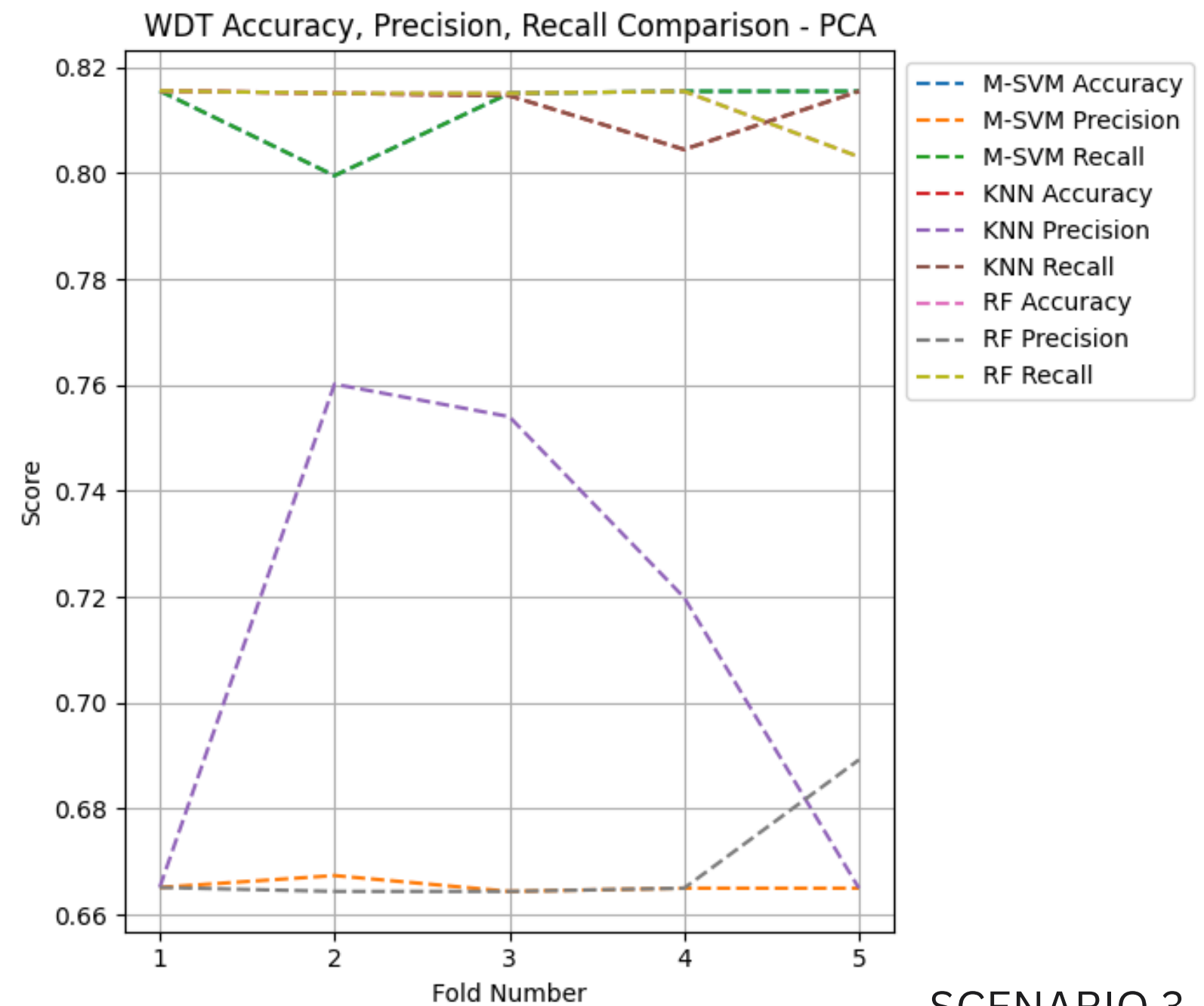
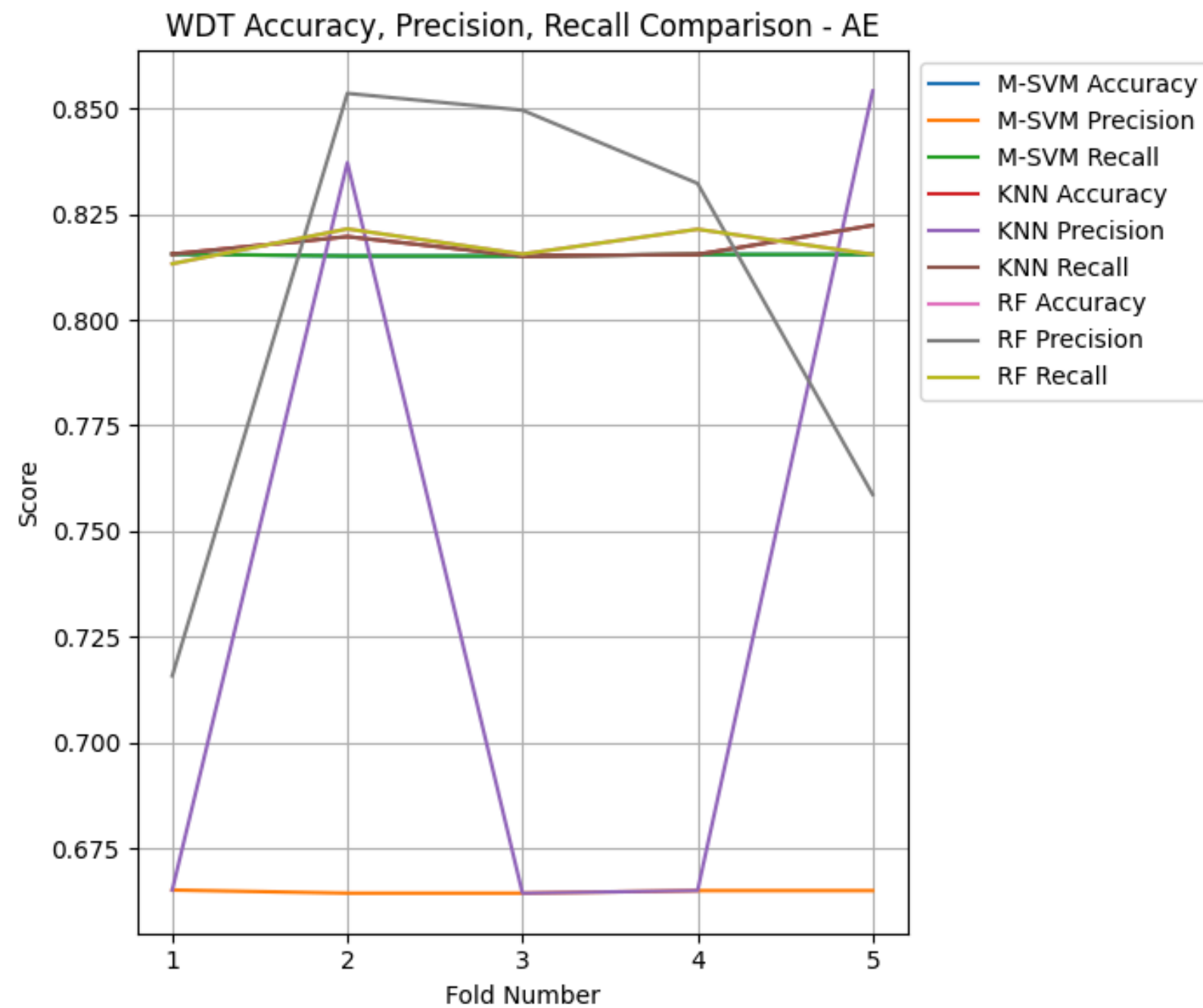
- ✓ **Scenario 1** → No exposure to attacks during training.
- ✓ **Scenario 2** → Tests generalization to an unseen attack.
- ✓ **Scenario 3** → Evaluates how models perform with minimal attack presence.



SCENARIO 2



- The WDT-AE has the most precision score
- Autoencoders performed well in unsupervised learning scenarios but required fine-tuning.

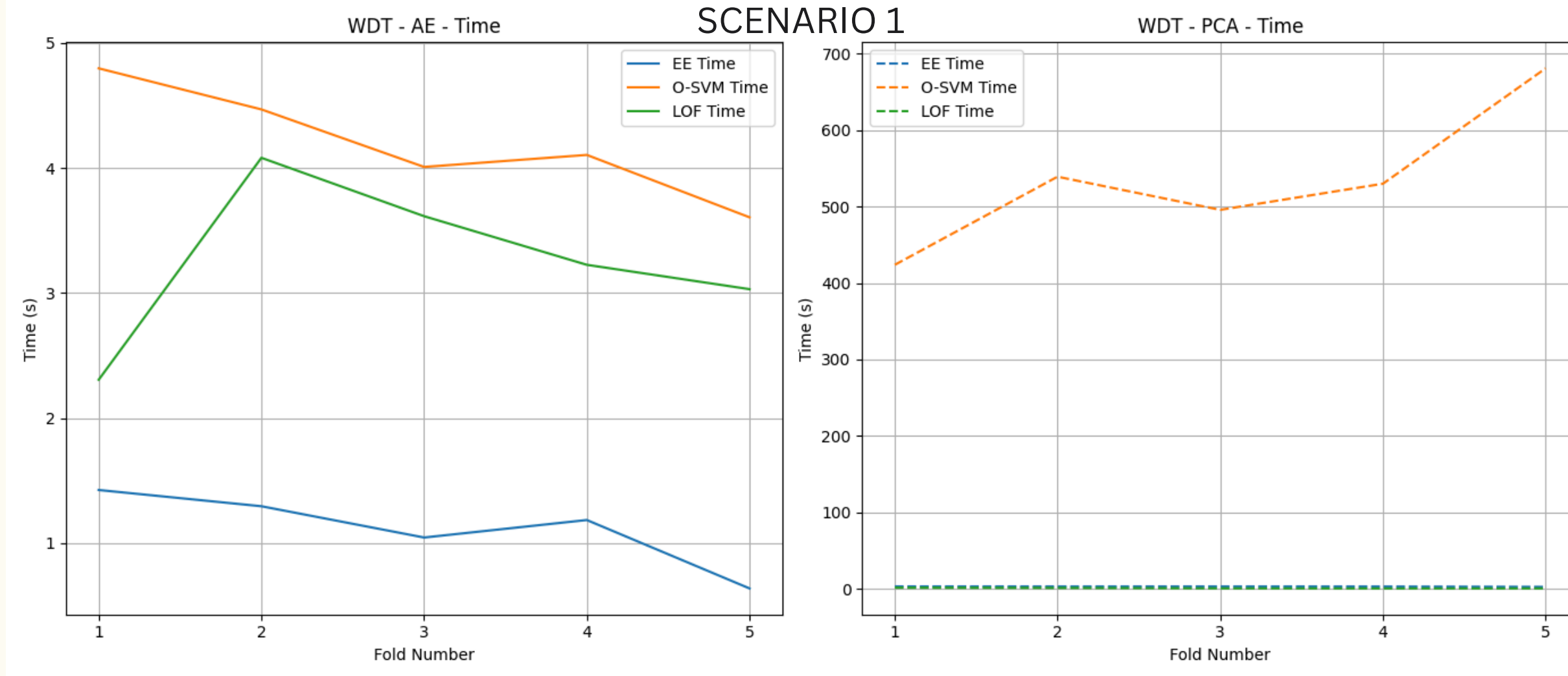


SCENARIO 3

- RF has the most precision and recall

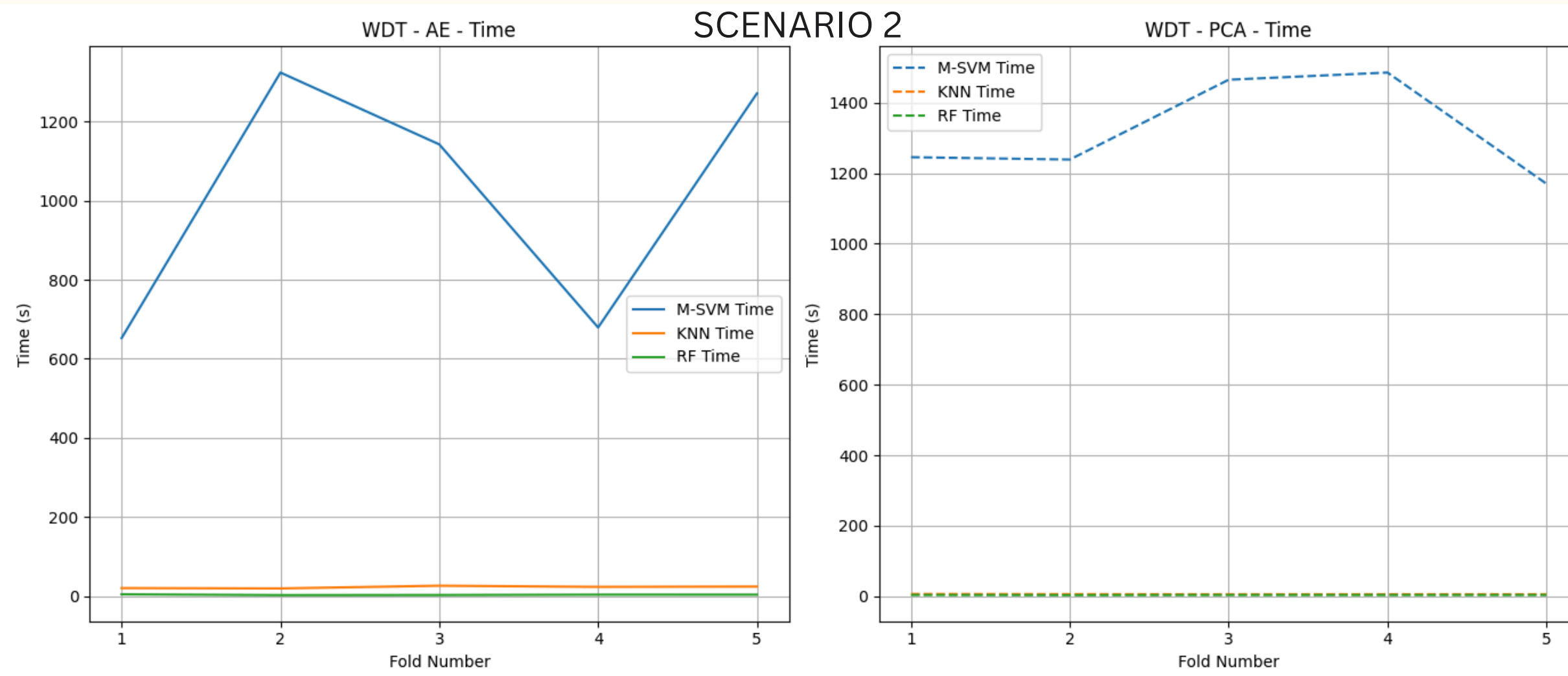
- ✓ **Scenario 1** → No exposure to attacks during training.
- ✓ **Scenario 2** → Tests generalization to an unseen attack.
- ✓ **Scenario 3** → Evaluates how models perform with minimal attack presence.

SCENARIO 1

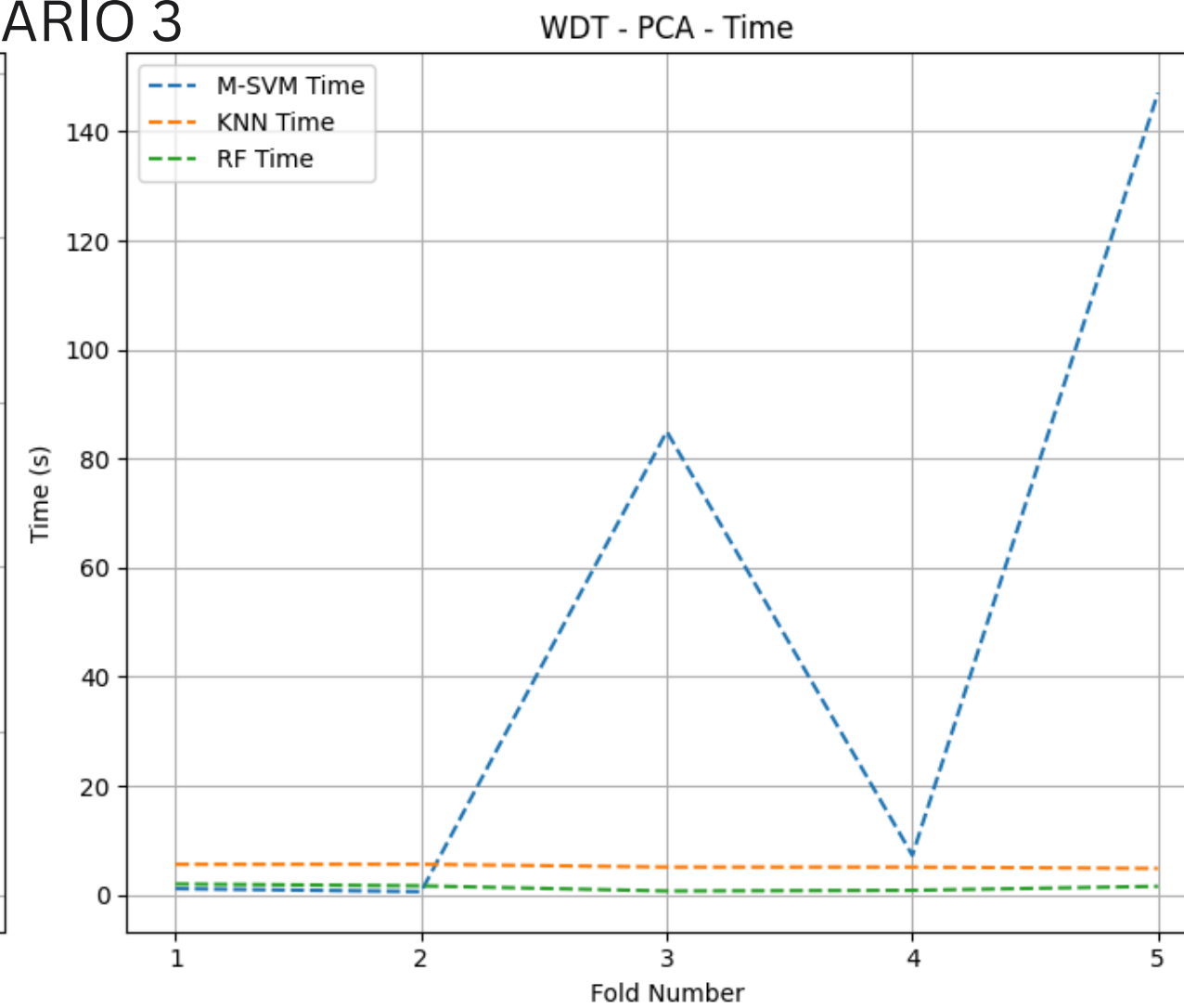
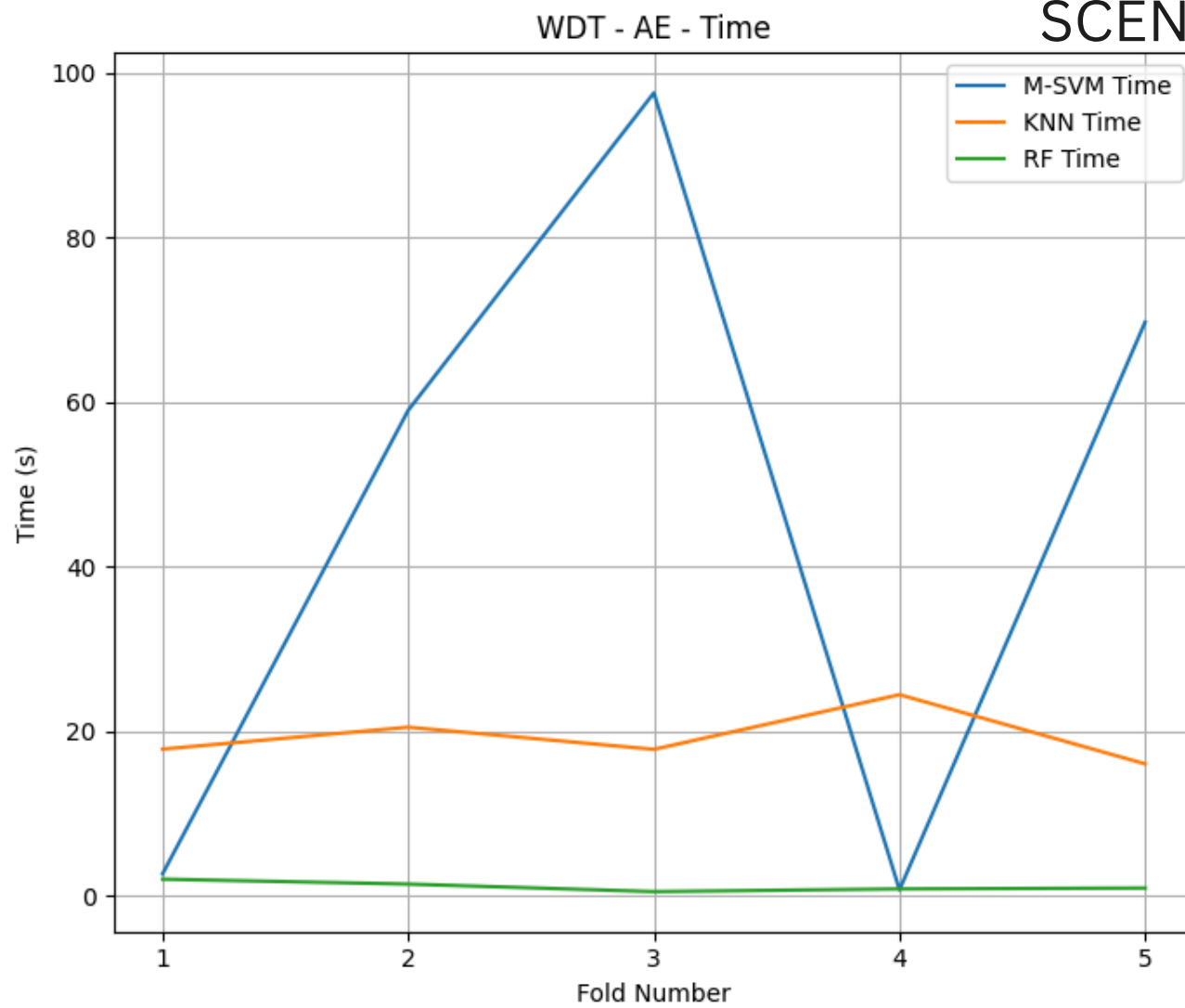


- A plot comparing execution times for different feature extraction and classification methods.
- It helps identify the most efficient models for real-time ICS anomaly detection.

SCENARIO 2

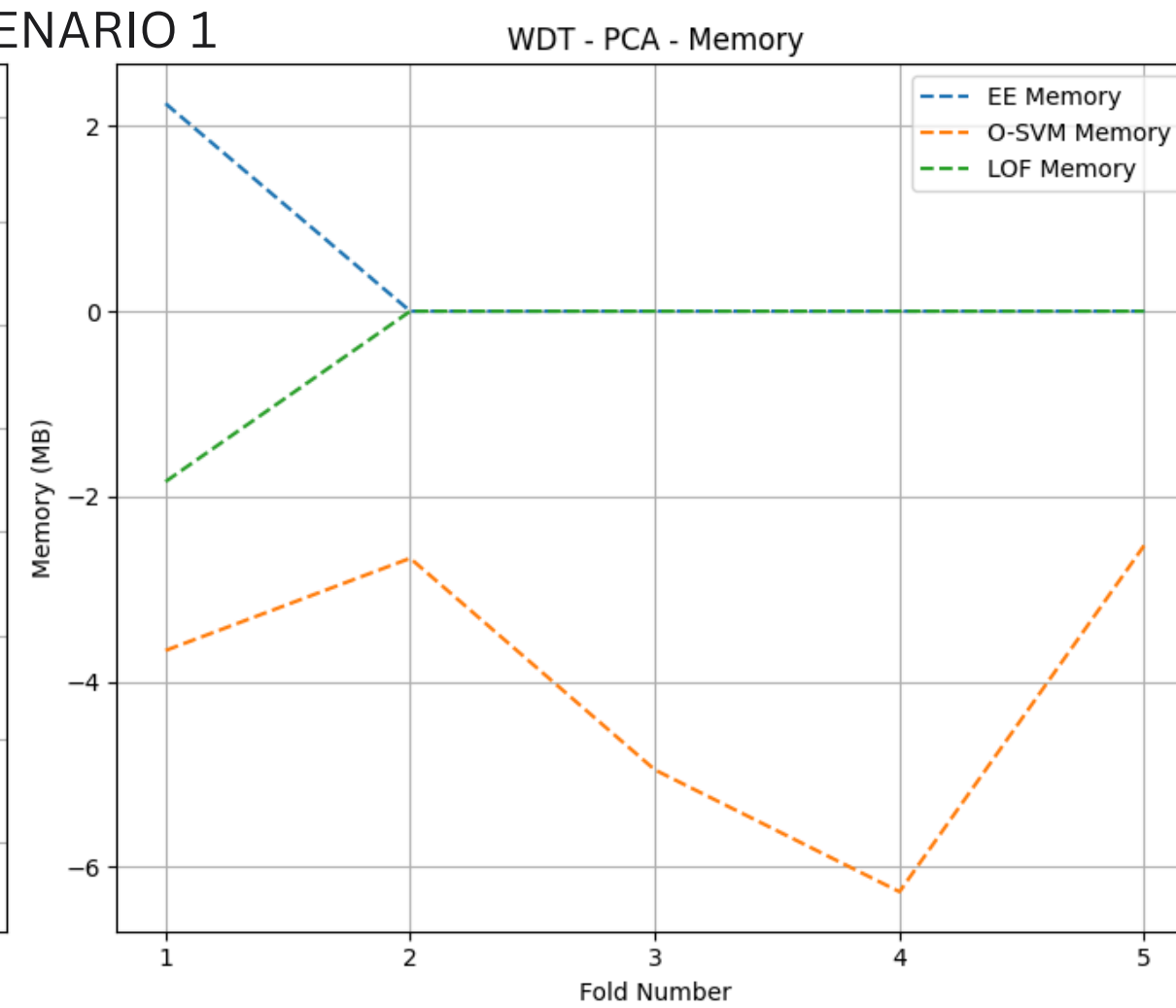
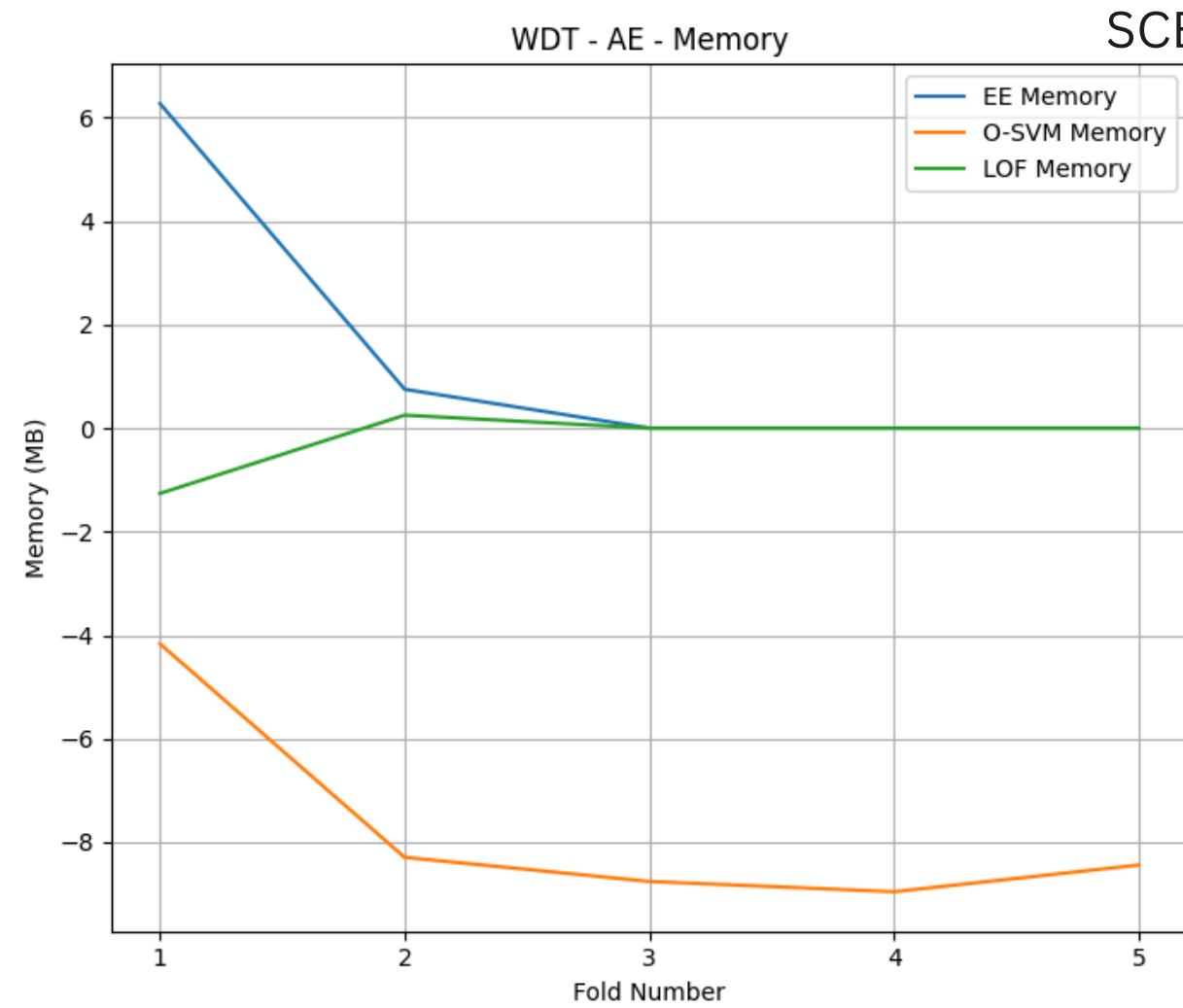


SCENARIO 3



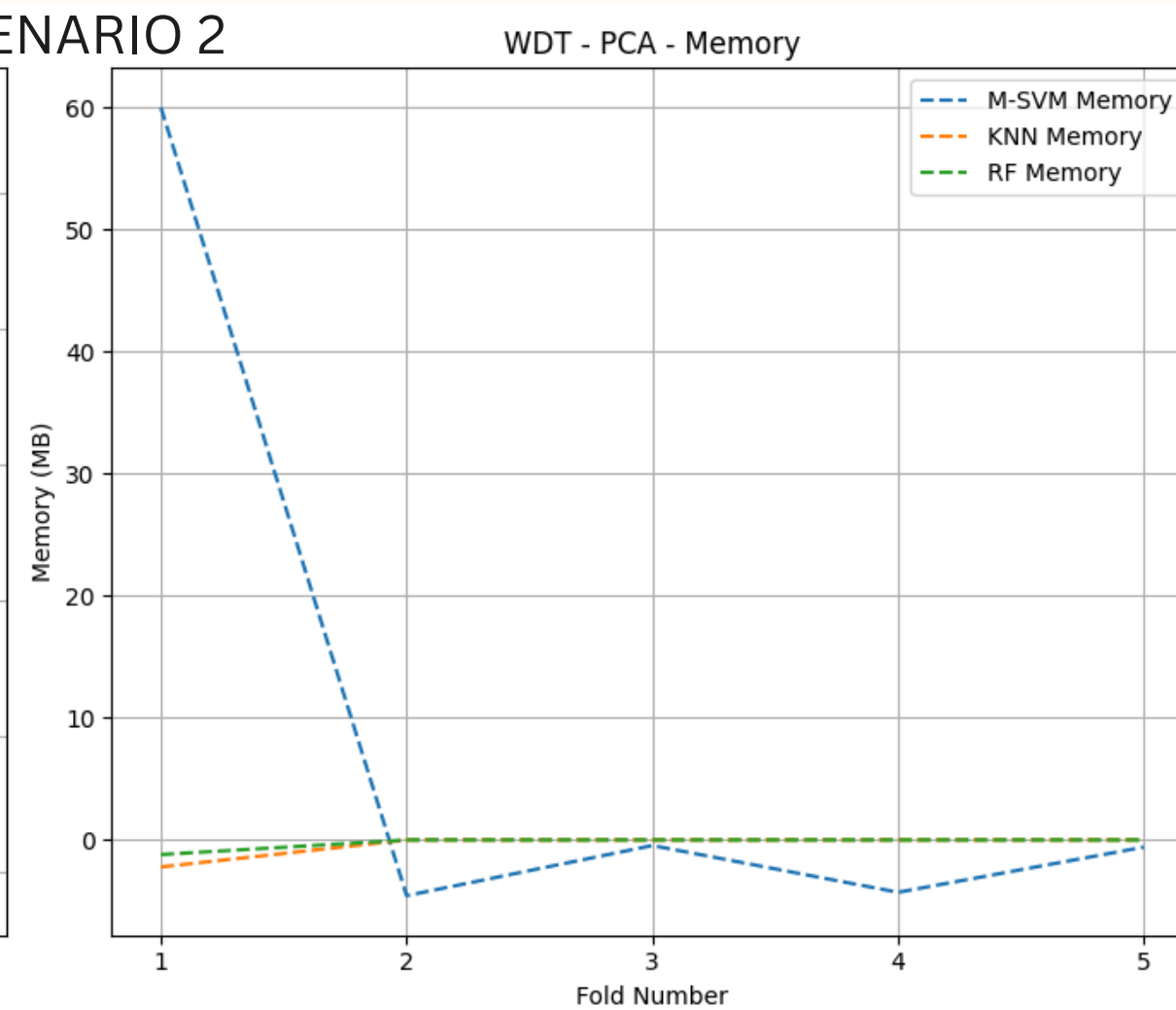
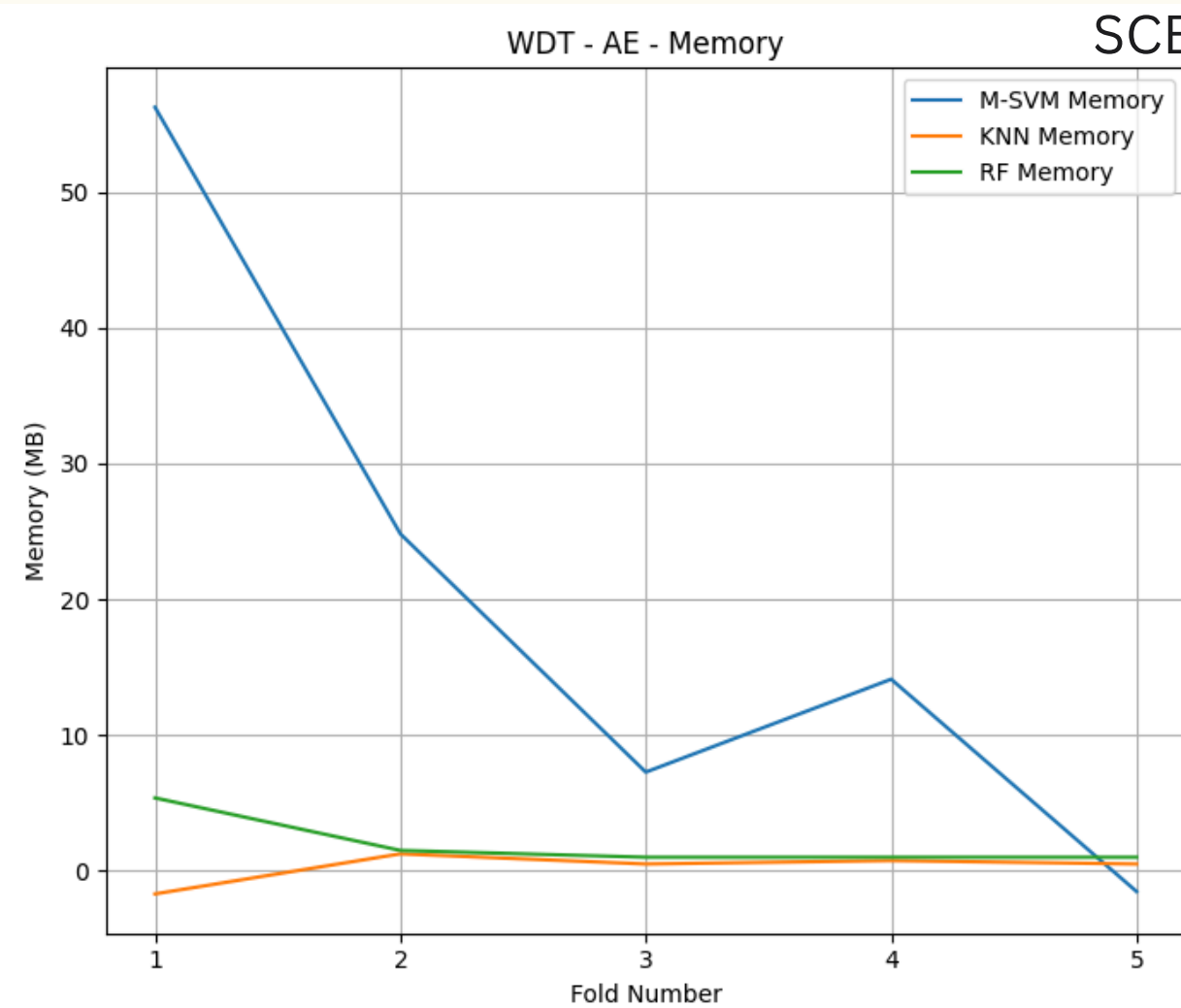
- A plot comparing execution times for different feature extraction and classification methods.
- It helps identify the most efficient models for real-time ICS anomaly detection.

SCENARIO 1

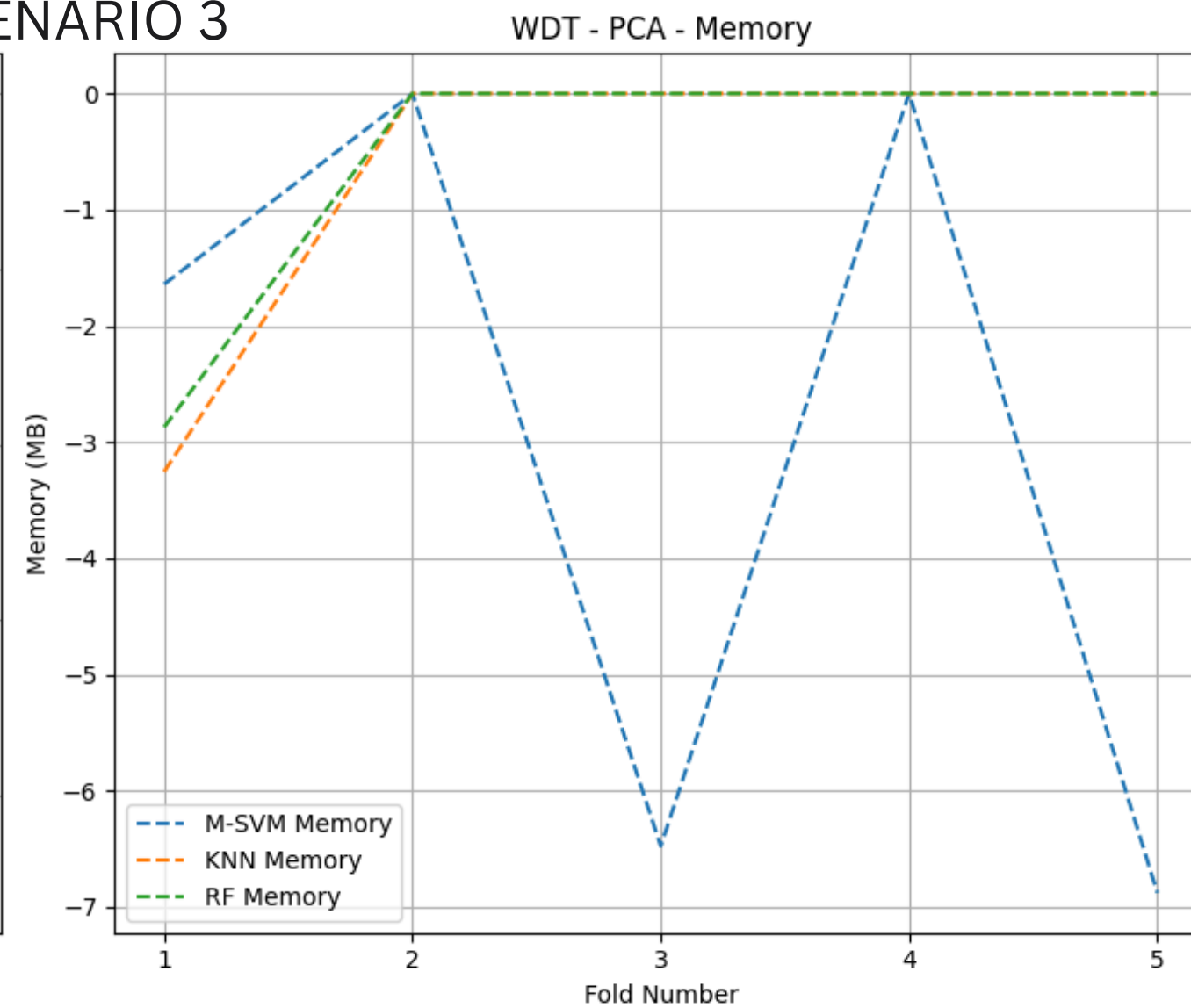
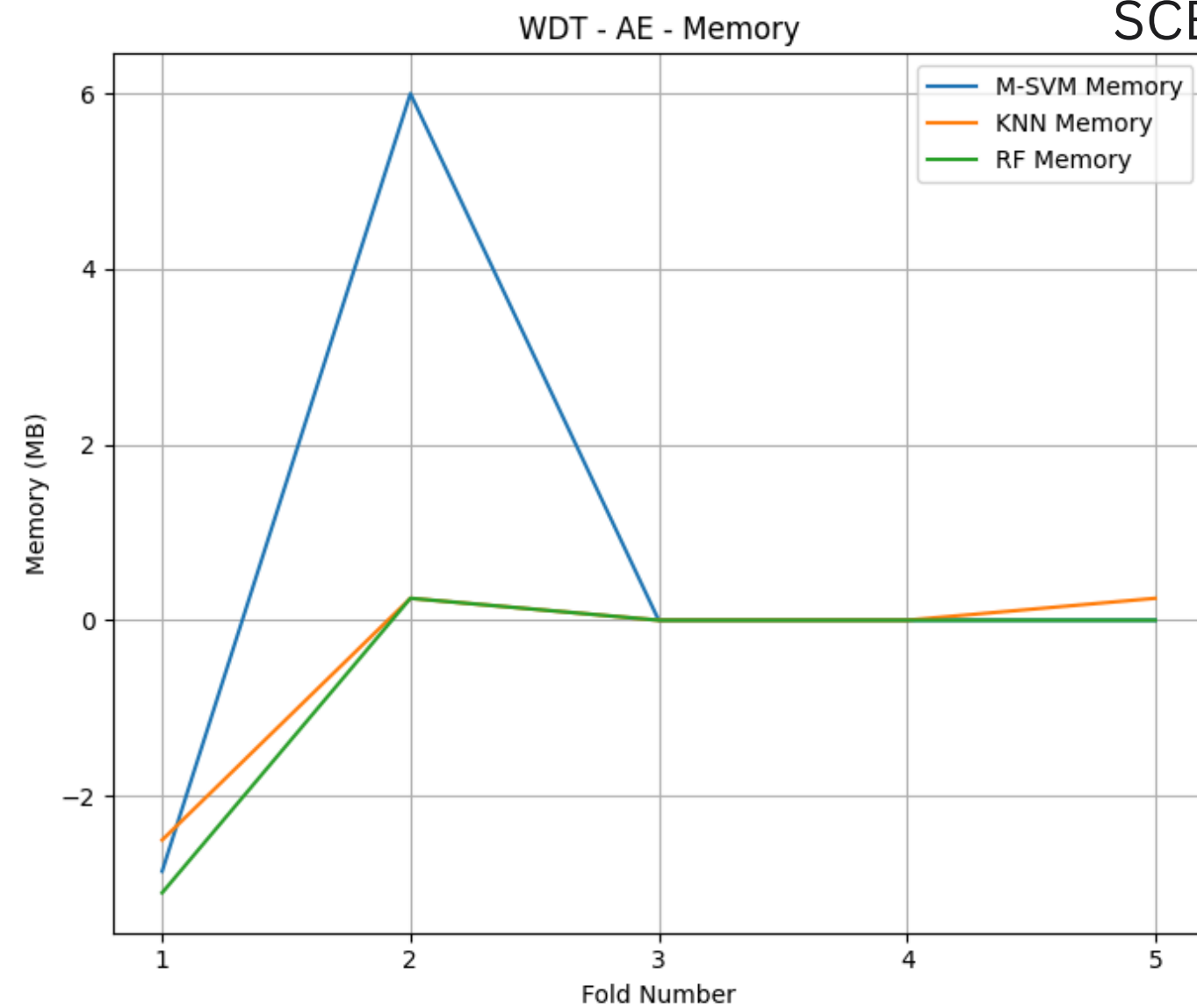


- A plot comparing memory consumption for different feature extraction and classification methods.
- It helps identify the most efficient models for real-time ICS anomaly detection.

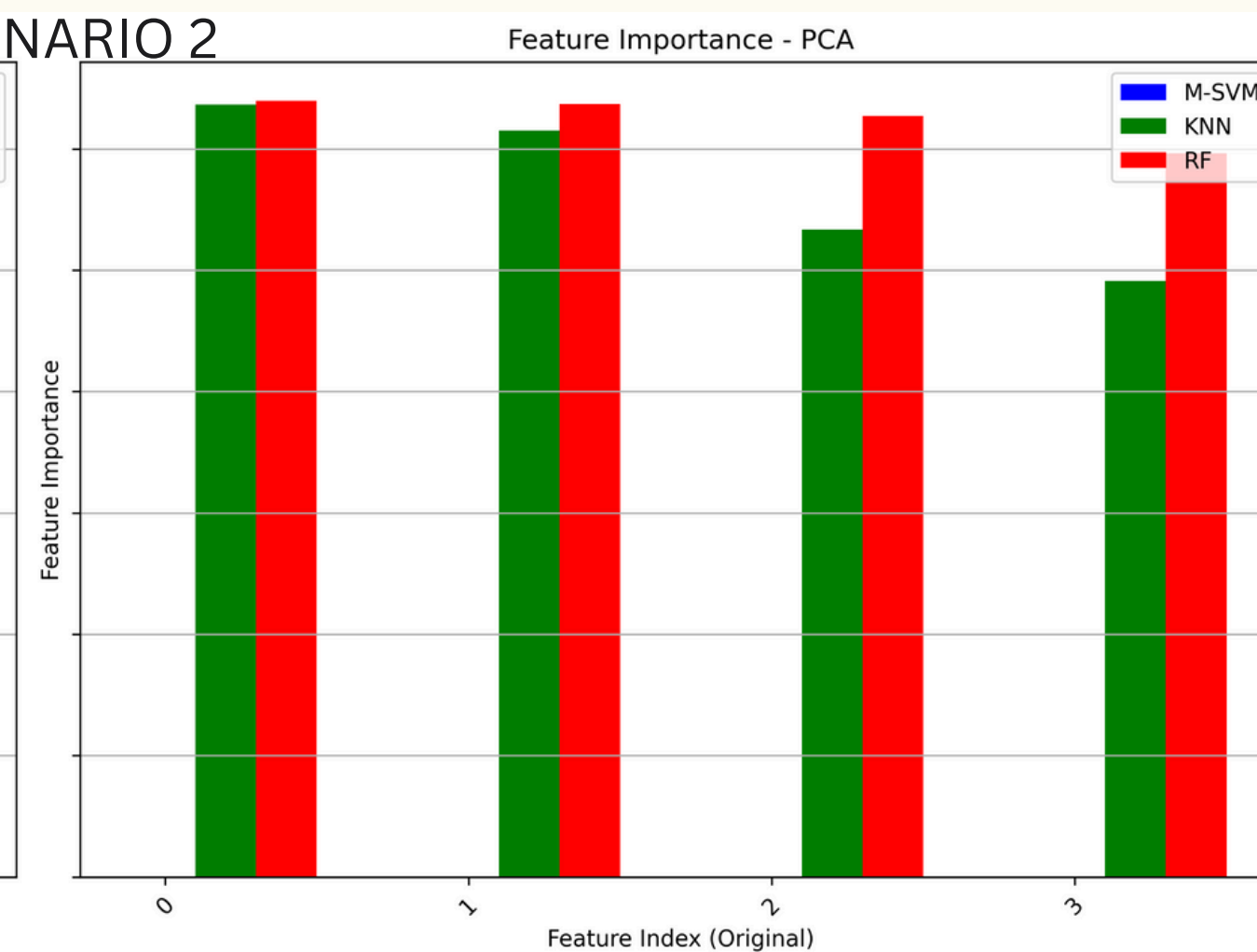
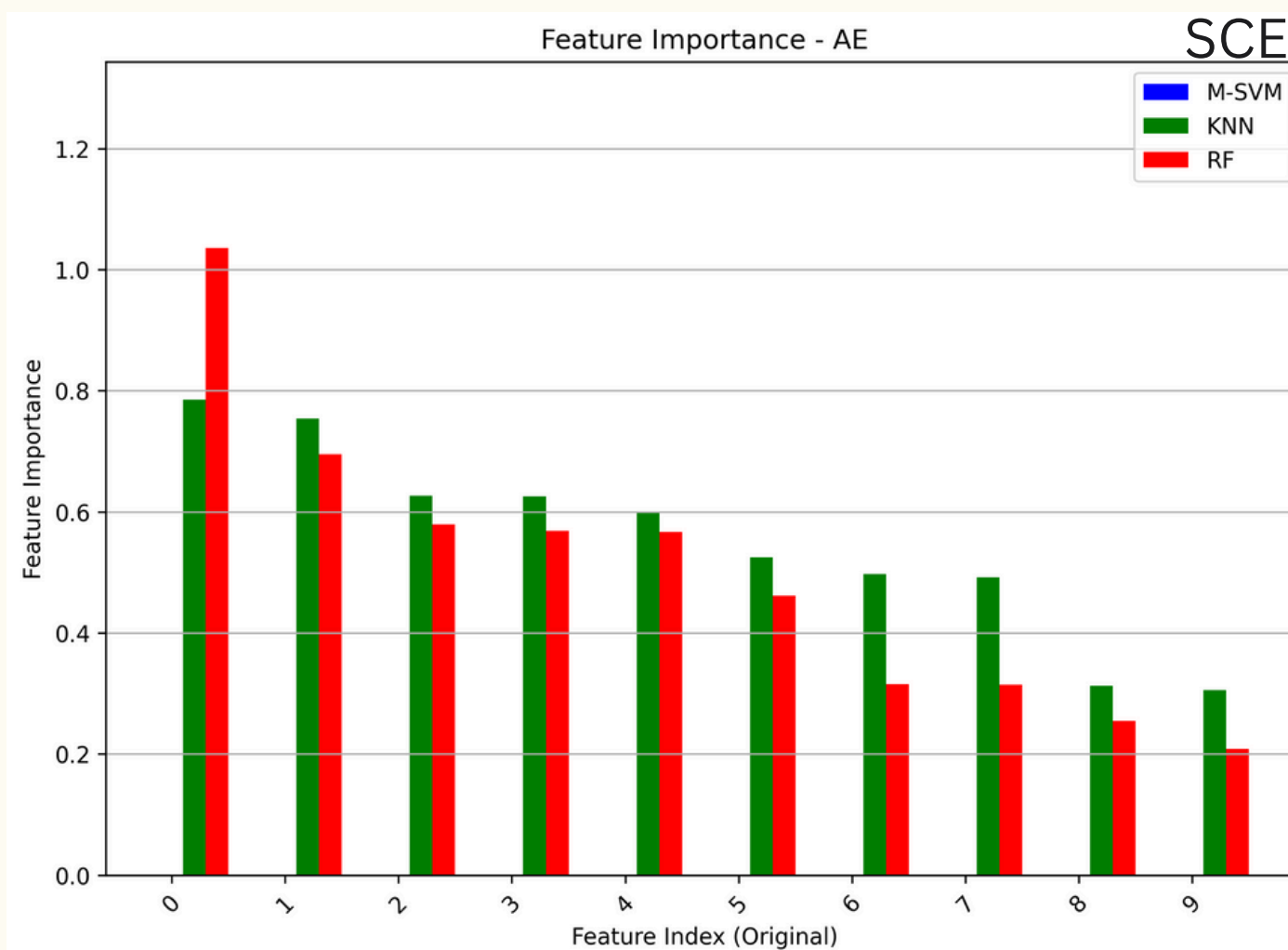
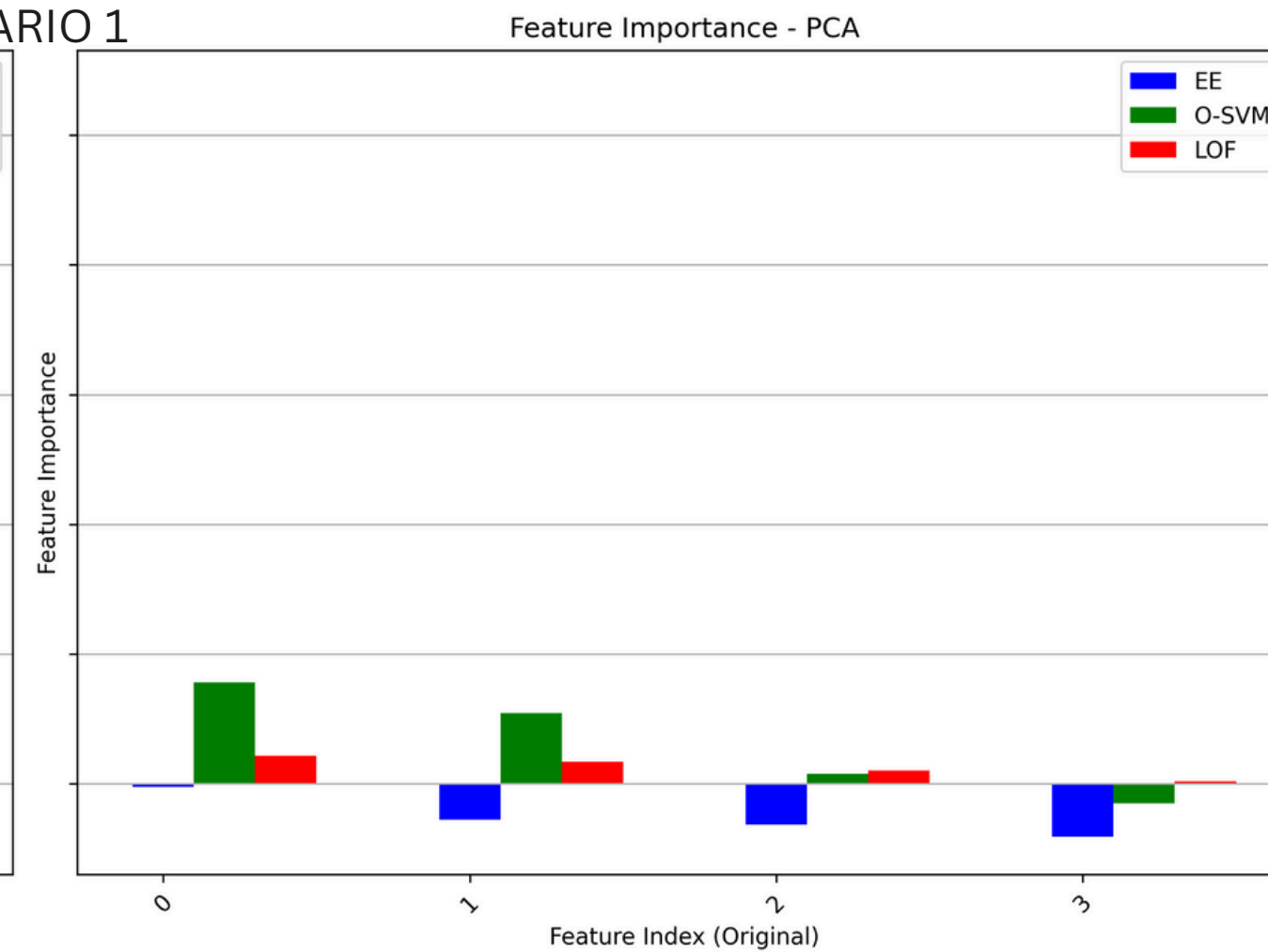
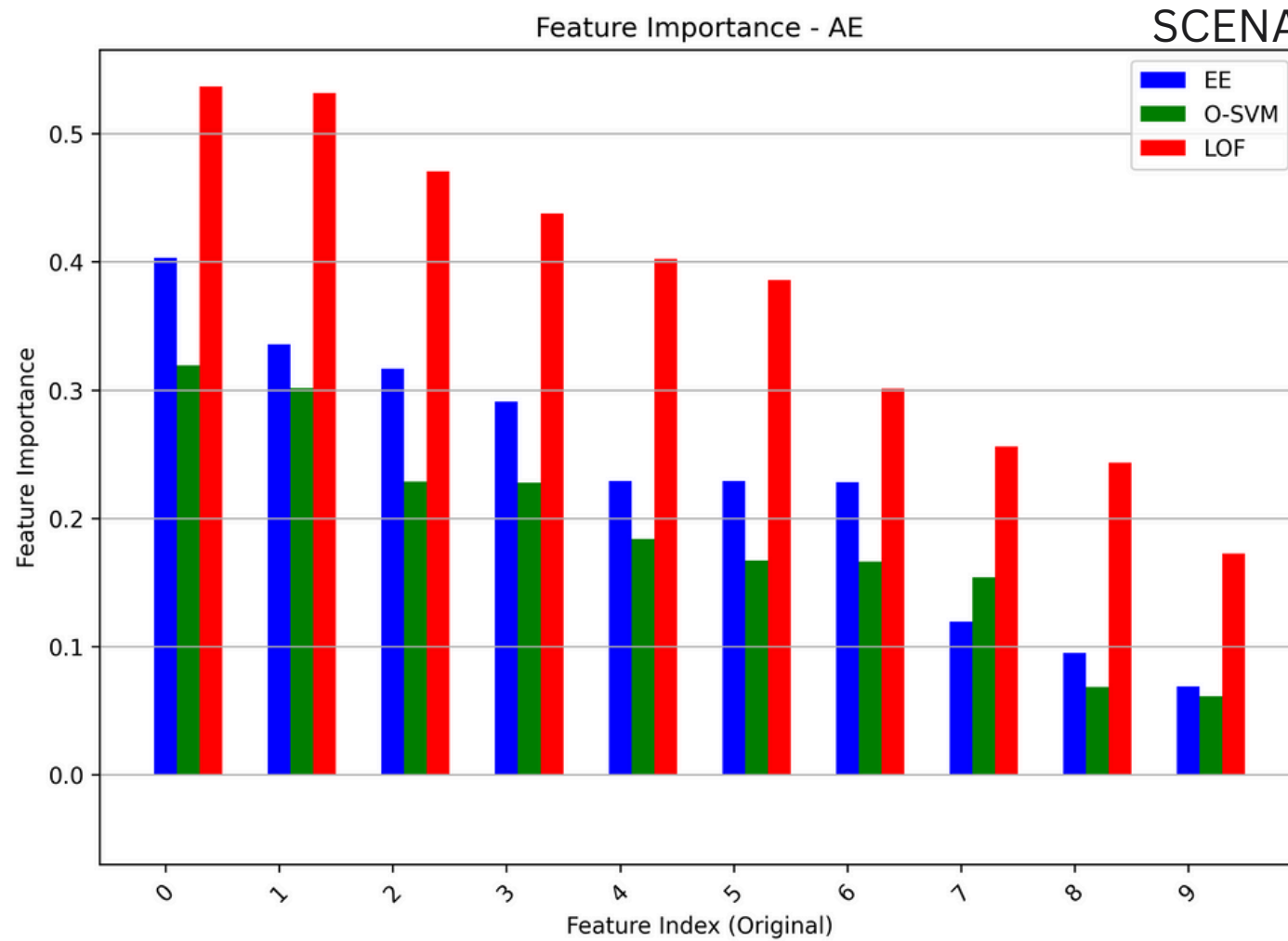
SCENARIO 2



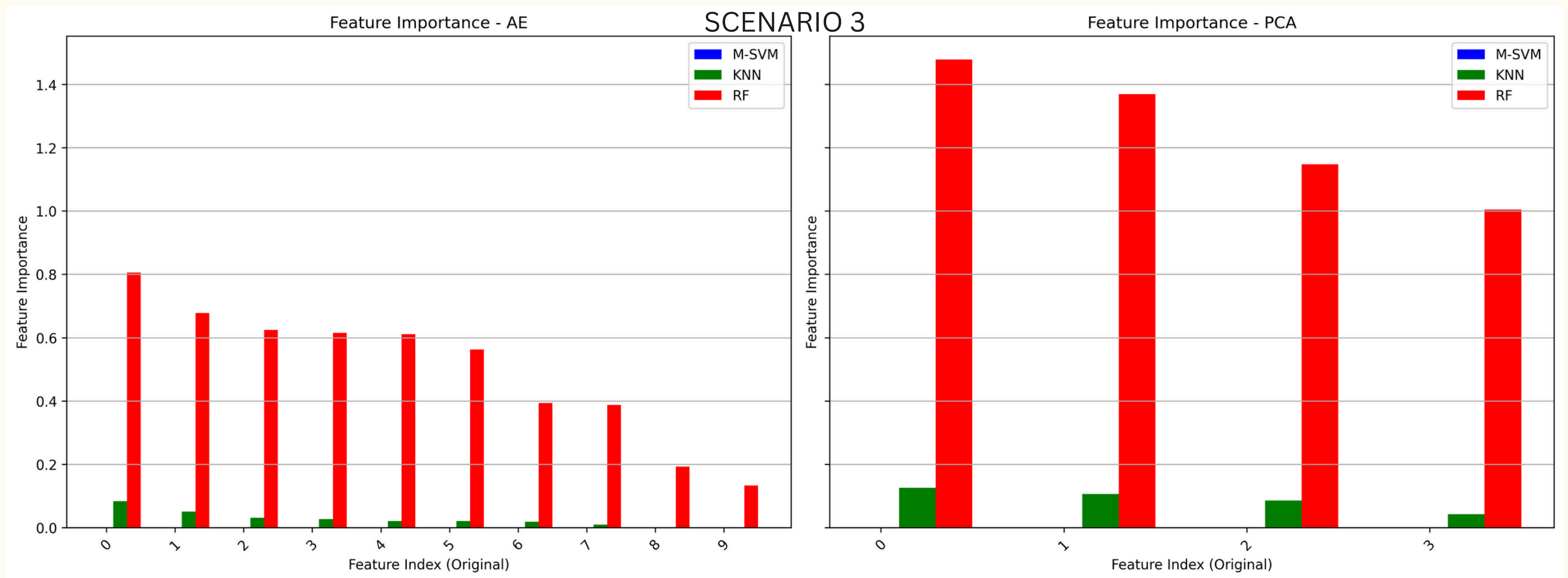
SCENARIO 3



- A plot comparing memory consumption for different feature extraction and classification methods.
- It helps identify the most efficient models for real-time ICS anomaly detection.

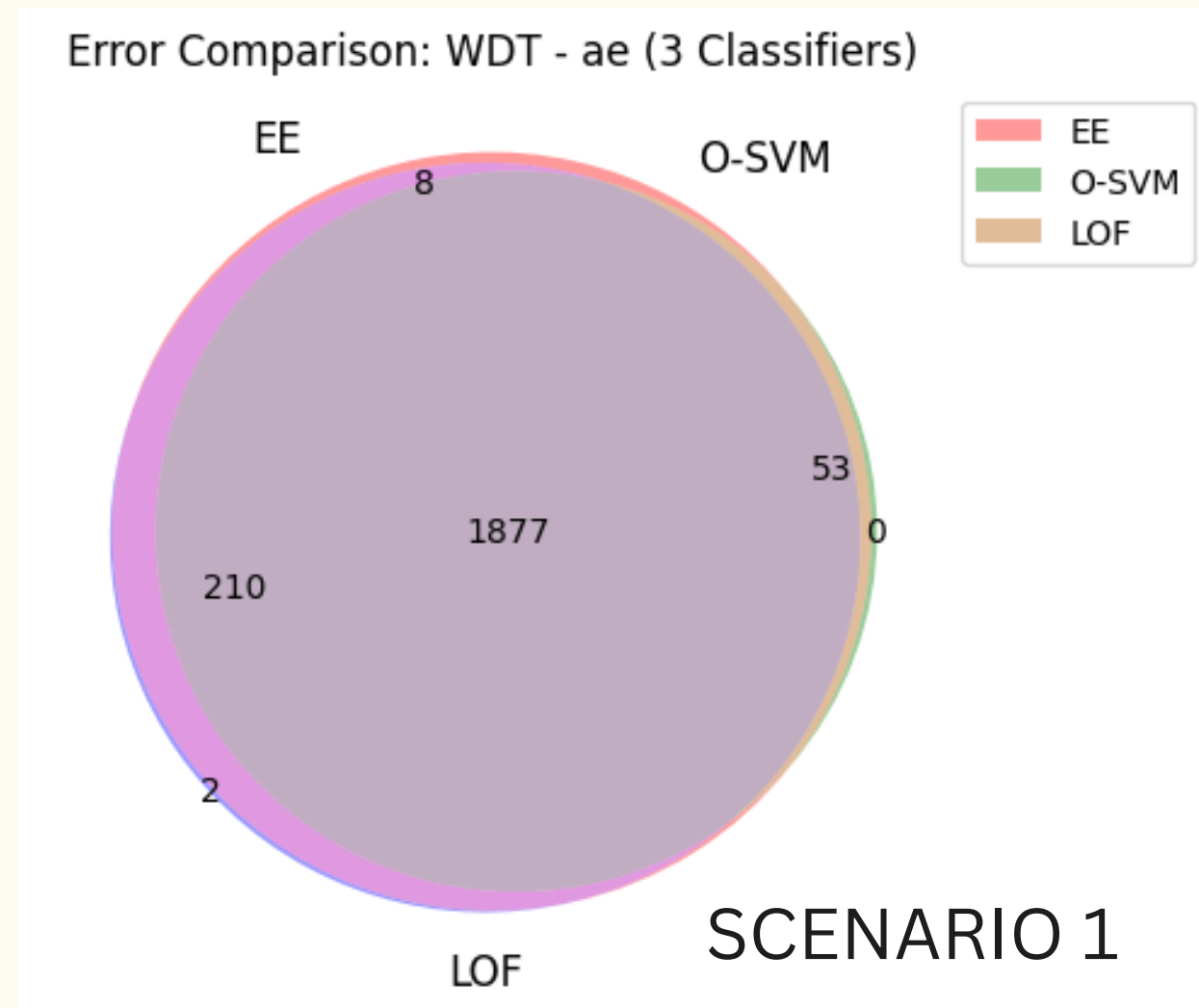


- A bar graph ranking the most influential features used in classification models.
- Identifying key features improves model interpretability and can help refine the dataset for better performance.

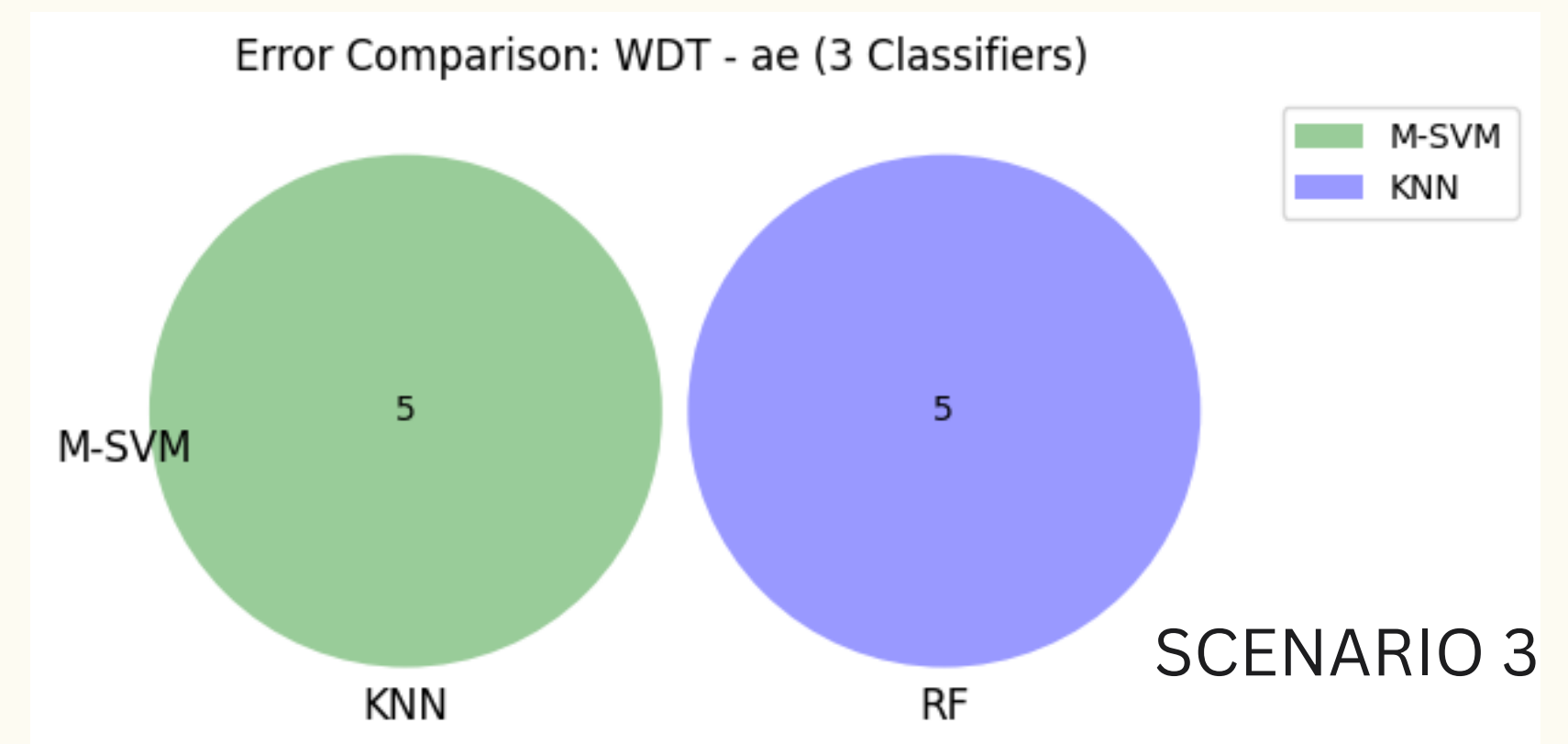
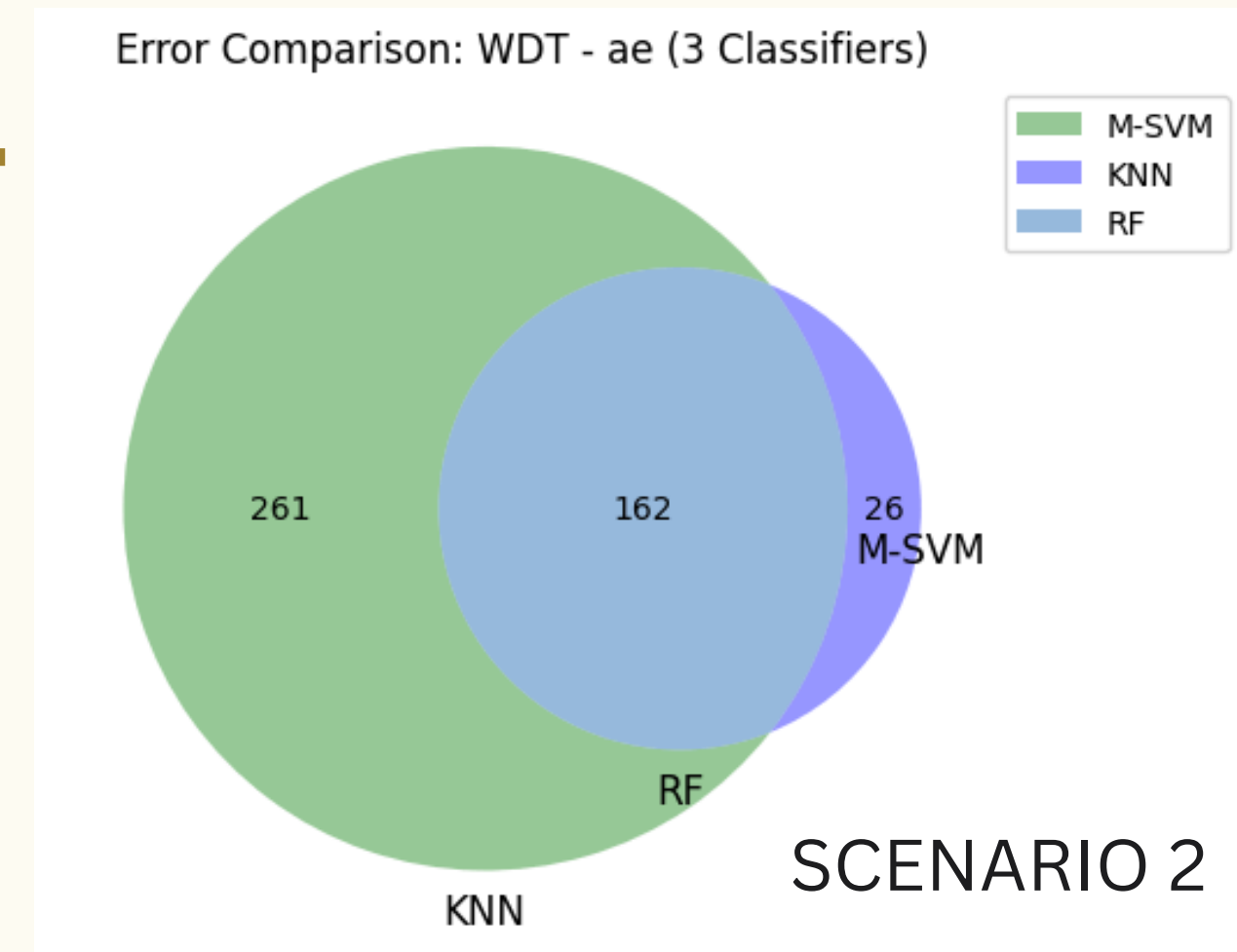


Feature importance plots for showing the most important features to the least important features

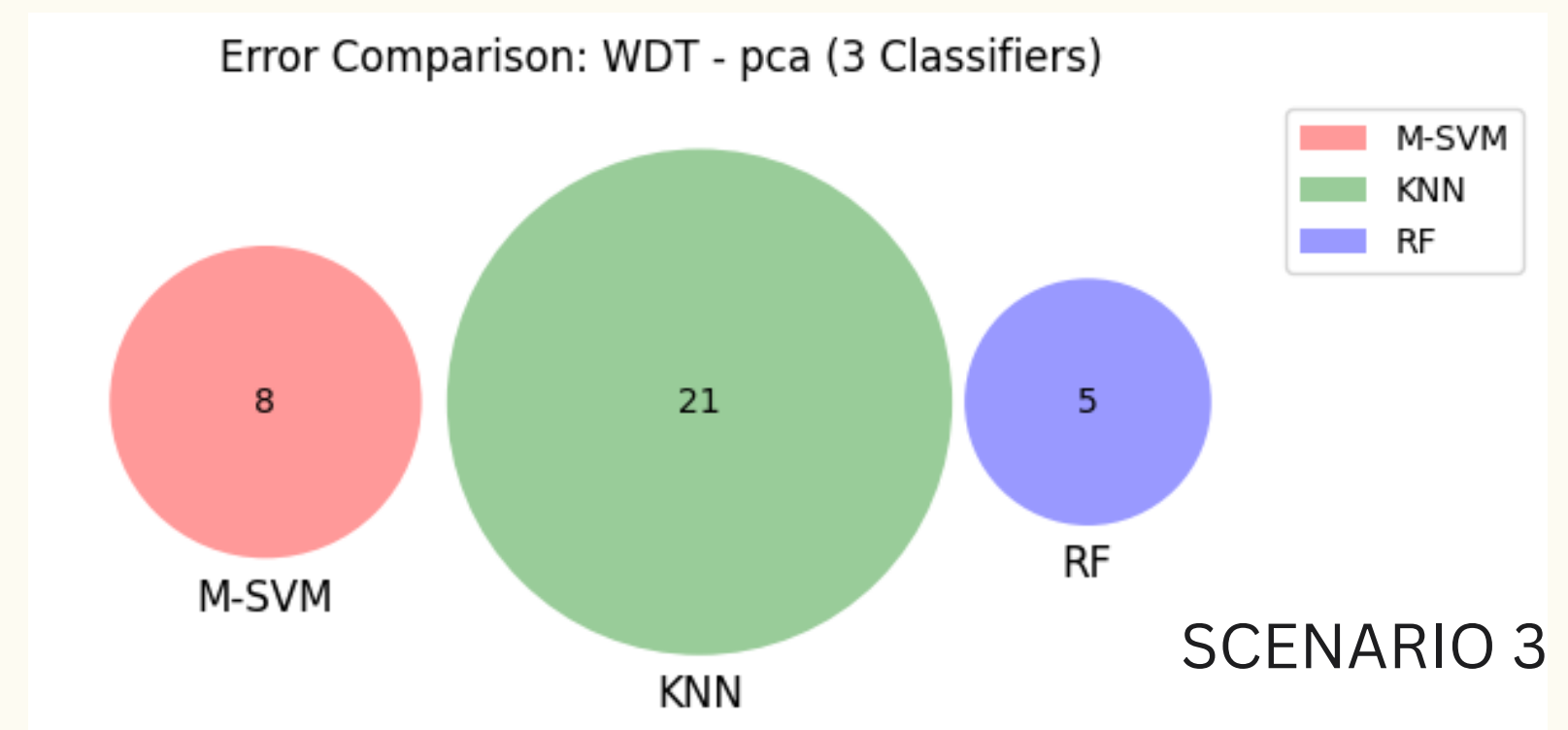
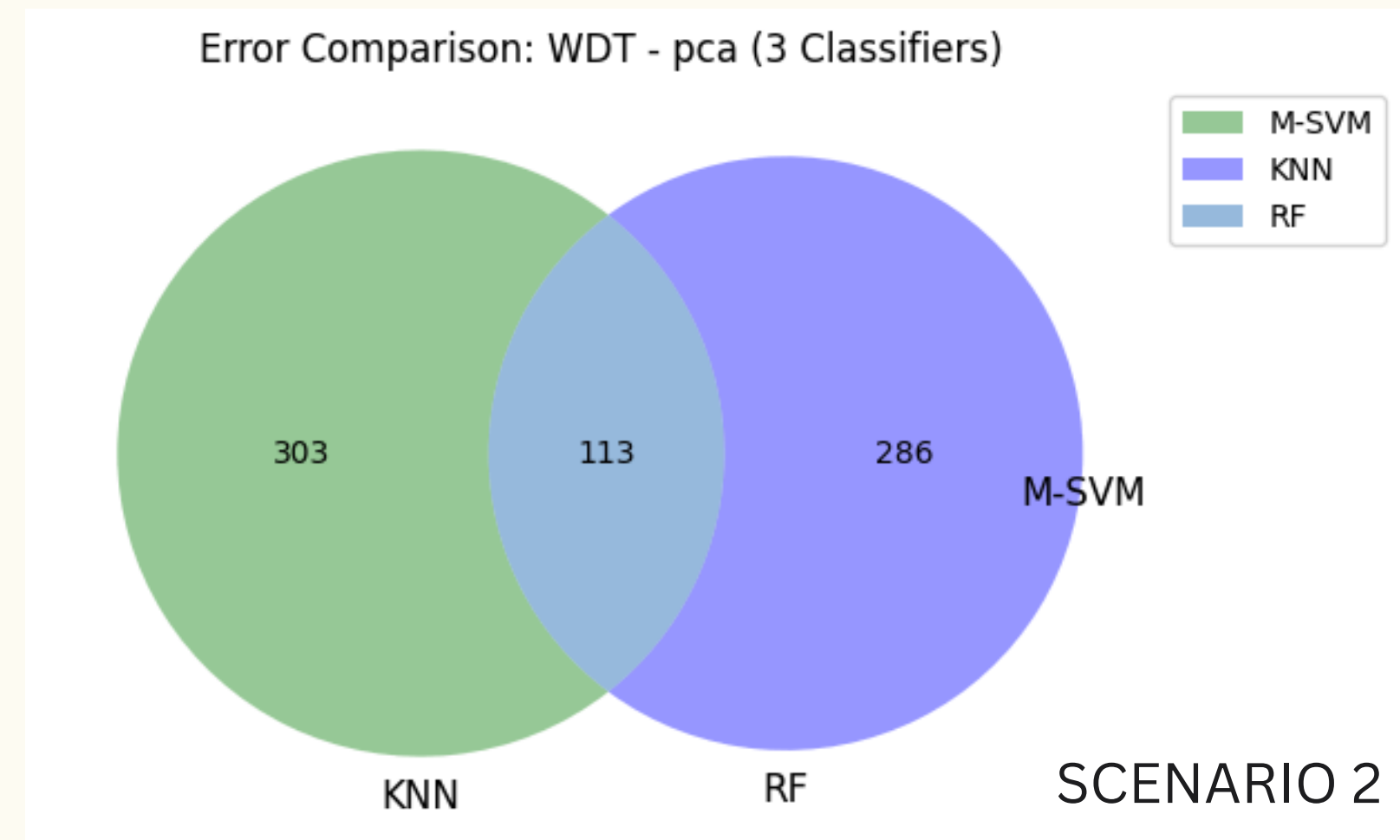
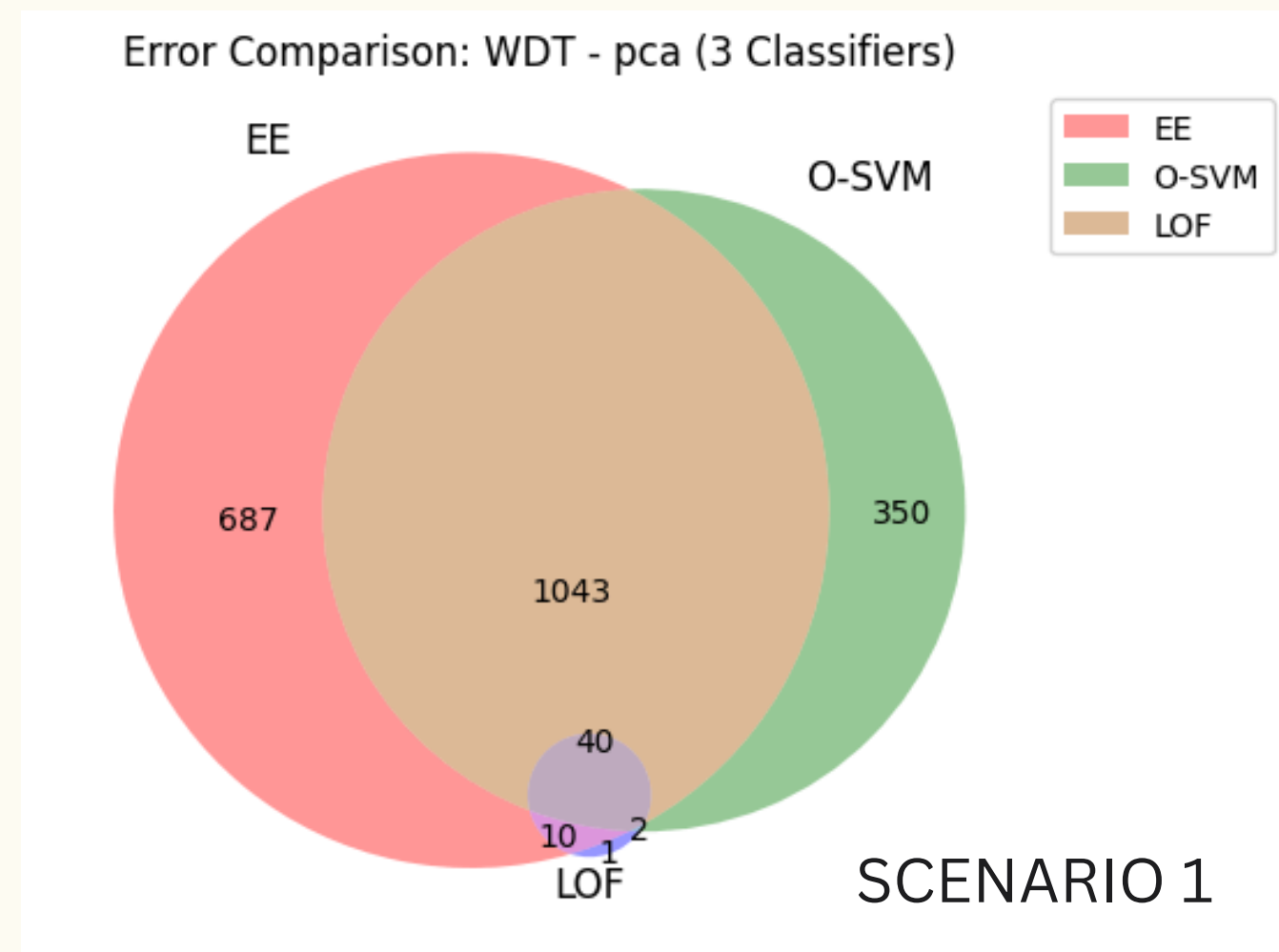
VENN - CLASSIFICATION ERRORS



- A Venn diagram showing misclassification overlaps between different ML models.
- It helps analyze which errors are common across multiple classifiers, indicating potential dataset issues or model weaknesses.



VENN - CLASSIFICATION ERRORS



- A Venn diagram showing misclassification overlaps between different ML models.
- It helps analyze which errors are common across multiple classifiers, indicating potential dataset issues or model weaknesses.

ENSEMBLE CLASSIFIER & DL-BASED CLASSIFIER

- 1** Deep Learning Classifier (CNN) – A Convolutional Neural Network (CNN) is trained on both sensor readings and network packets to detect anomalies.
- 2** Ensemble Classifier – Combines multiple ML models to improve detection performance using three voting strategies:
 - Random Voting → Picks a prediction randomly from the ensemble models.
 - Majority Voting → Takes the most common prediction among models.
 - All Voting → Flags an anomaly only if all models agree.

ENSEMBLE CLASSIFIER & DL-BASED CLASSIFIER

Single-Class:

- ◆ *One-class SVM*
- ◆ *Local Outlier Factor (LOF)*
- ◆ *Elliptic Envelope*

Multi-Class:

- ◆ Random Forest
- ◆ k-Nearest Neighbors (k-NN)
- ◆ Multi-class SVM

WHY IS THIS IMPORTANT?

- ◆ CNNs can detect complex patterns that traditional models miss.
- ◆ Ensemble methods improve robustness by combining multiple models.

ENSEMBLE CLASSIFIER & DL-BASED CLASSIFIER

◆ Scenario 1: Ensemble Classifier, Autoencoder as classifier and Statistical methods

📌 Objective: Evaluate ensemble classifiers and statistical methods on parsed physical readings and network packets.

📌 Evaluation:

✓ Ensemble Methods: Compare performance across different ensemble classifiers.

✓ Feature Sets: PCA (sensor data), Autoencoder (sensor data), Autoencoder (network data).

✓ Statistical Methods: Evaluate CUSUM & EWMA on parsed physical readings.

✓ Autoencoder as Classifier: Test on both sensor and network data.

📌 Key Question: Which ensemble classifier and statistical method provide the best precision and recall?

◆ Scenario 2: Deep Learning Classifier Evaluation

📌 Objective: Compare deep learning-based classifiers on parsed physical readings and network packets.

📌 Evaluation:

✓ Train & Test Deep Learning Classifier separately for both data types.

✓ Optimize hyperparameters for best performance.

📌 Key Question: How effective are deep learning models in detecting cyber-physical anomalies?

◆ Scenario 3: Deep Learning Classifier in Different Conditions

📌 Objective: Assess deep learning classifier performance under new evaluation conditions.

📌 Evaluation:

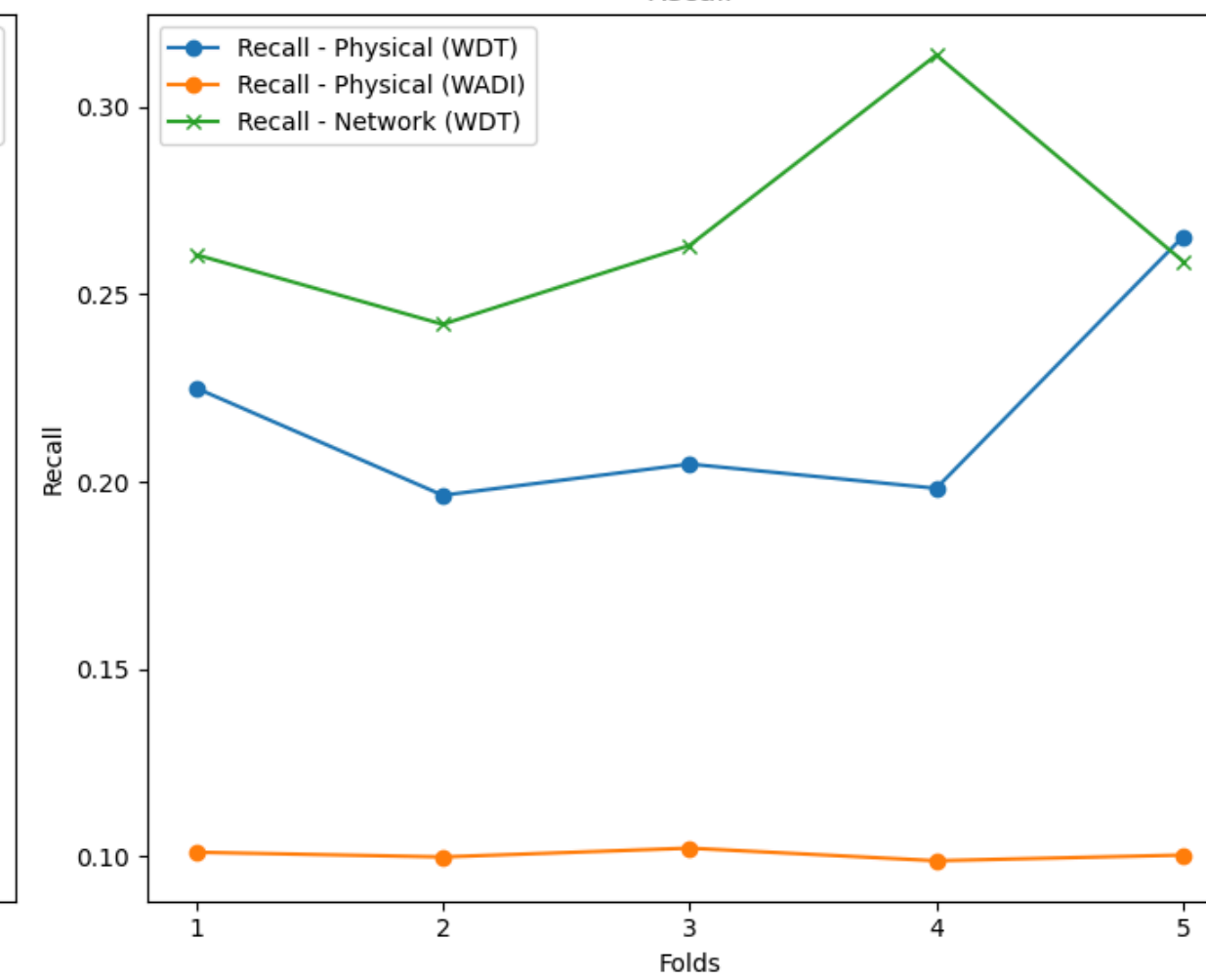
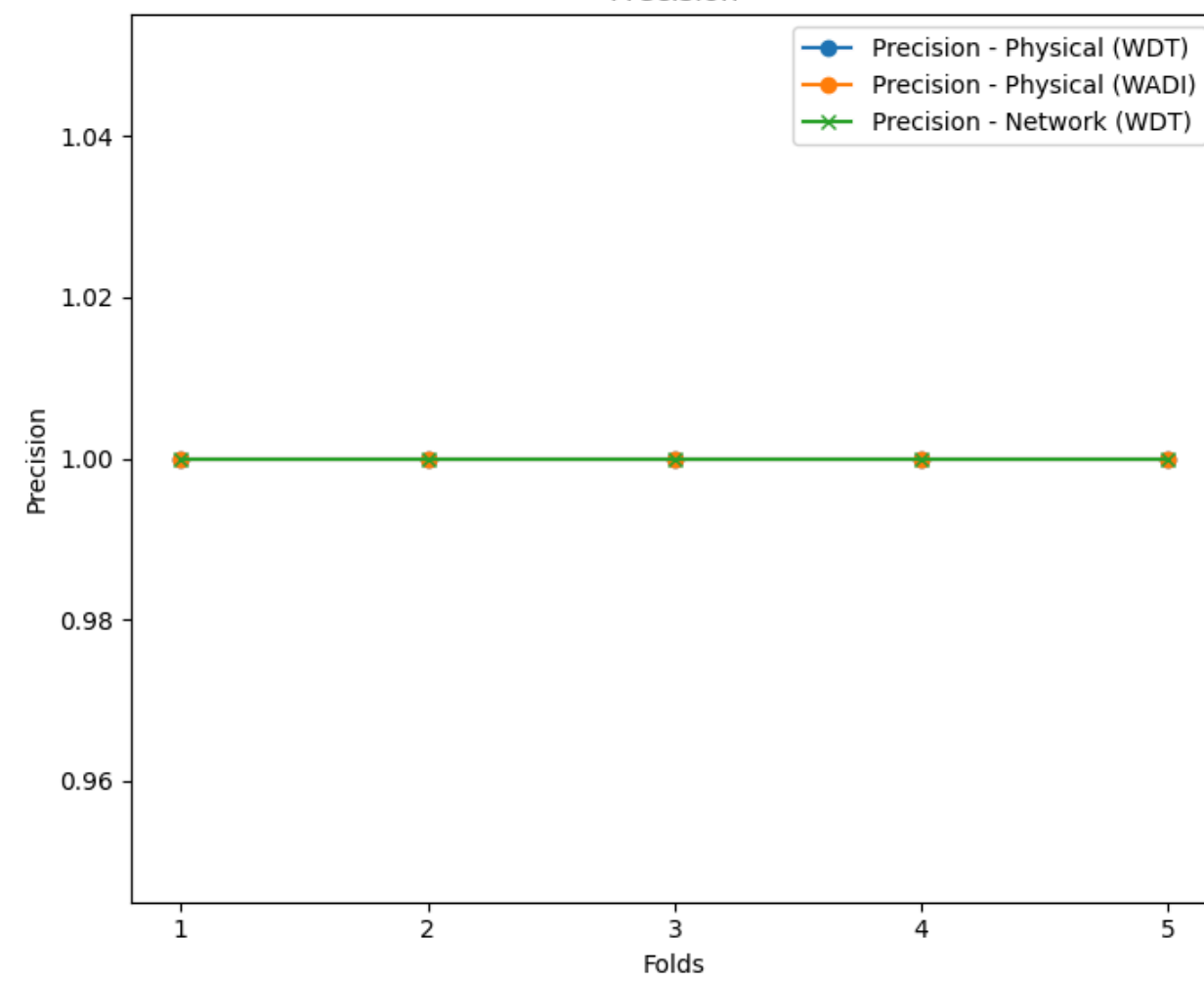
✓ Train & Test Deep Learning Classifier separately for sensor and network data under new conditions.

✓ Optimize hyperparameters to ensure high precision and recall.

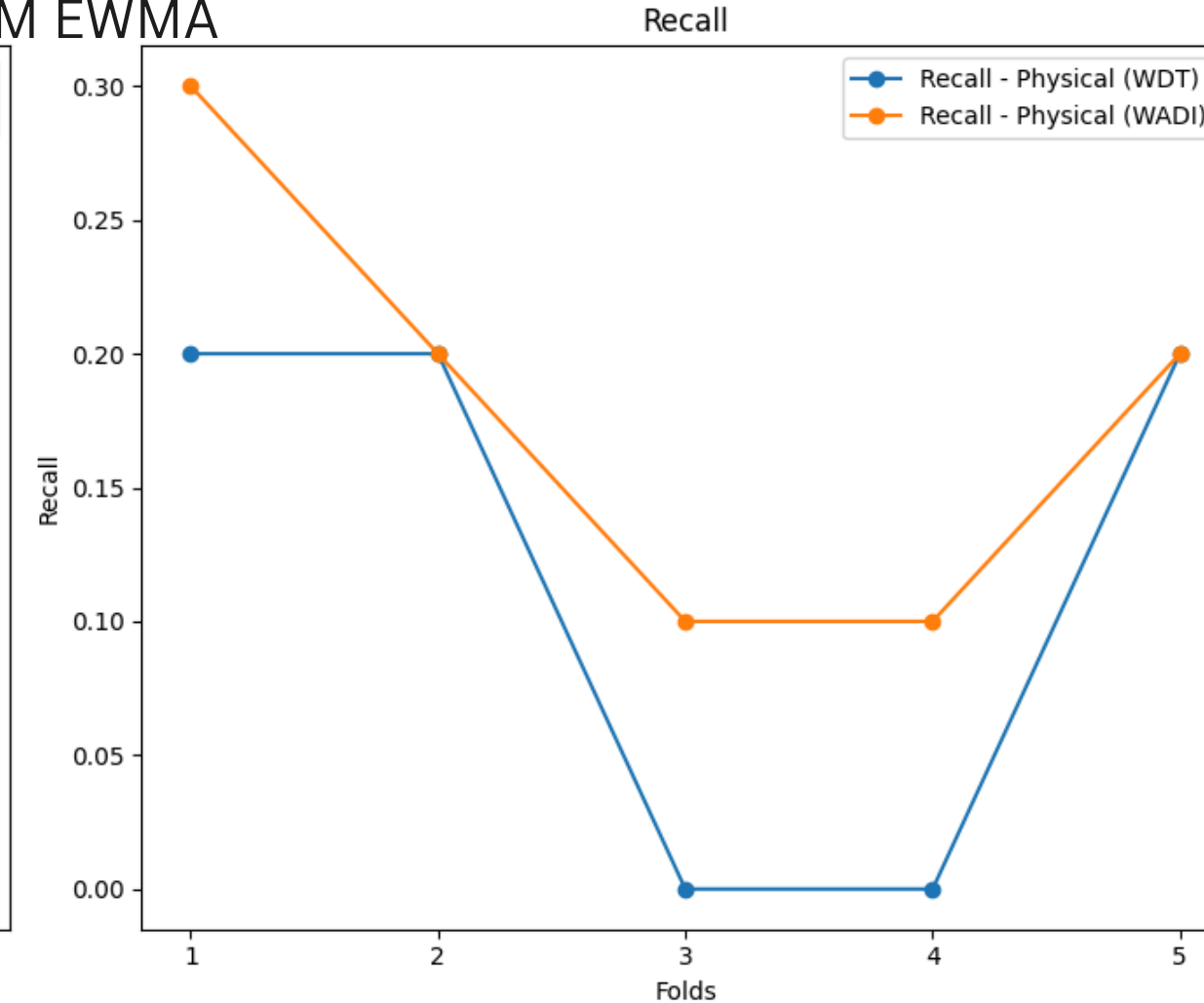
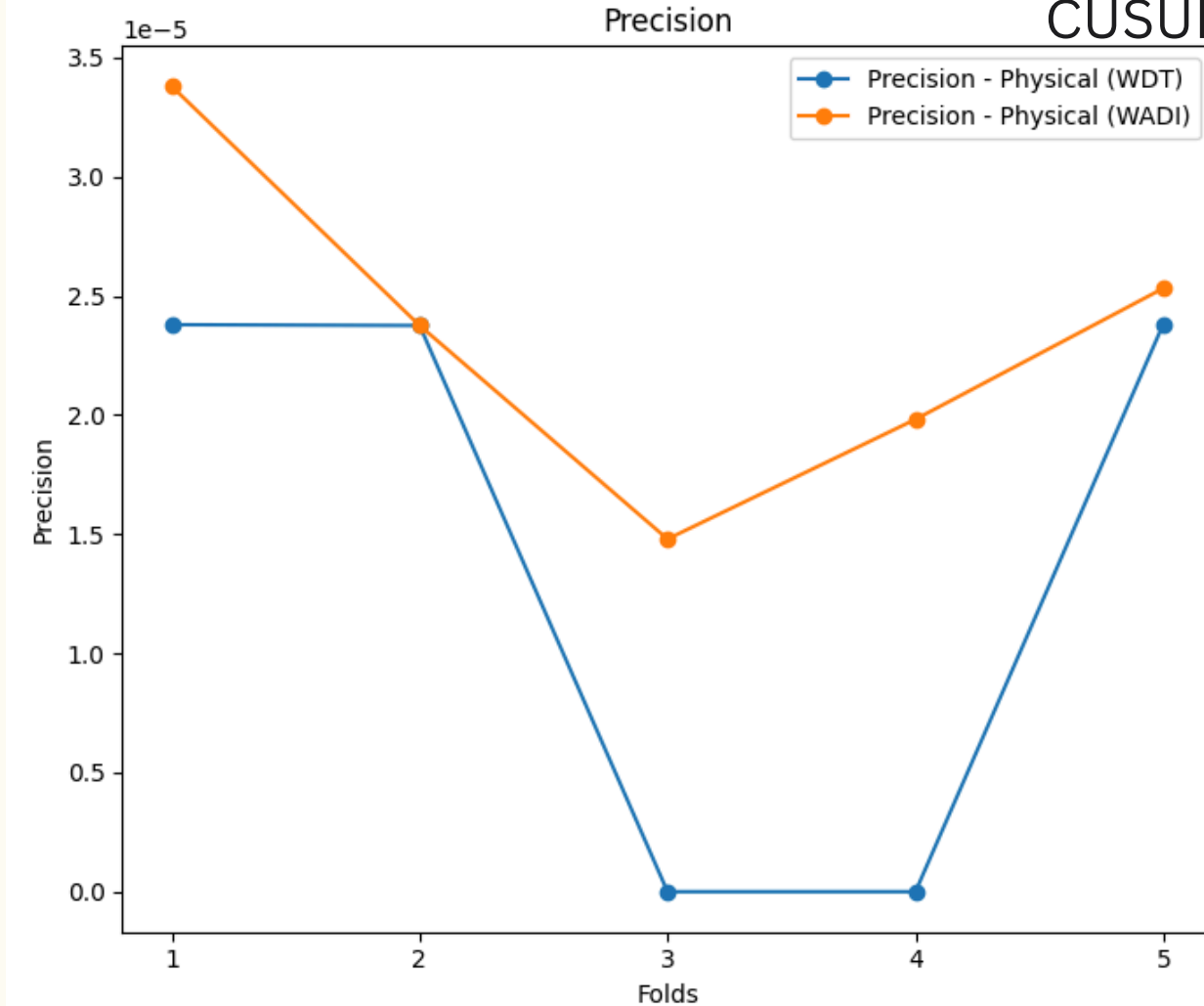
📌 Key Question: Can deep learning classifiers adapt to different attack scenarios effectively?

RESULTS

Autoencoder as classifier

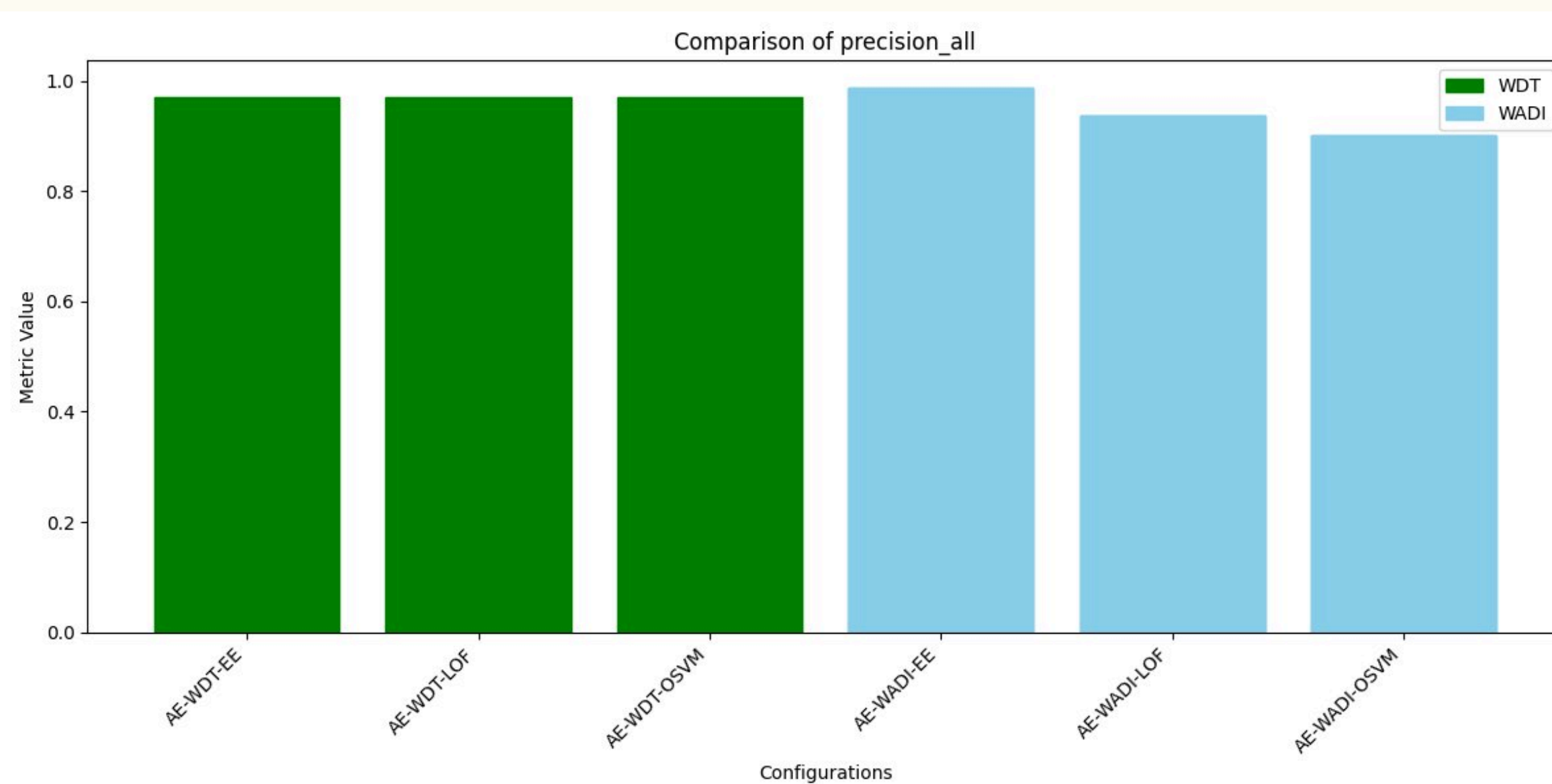
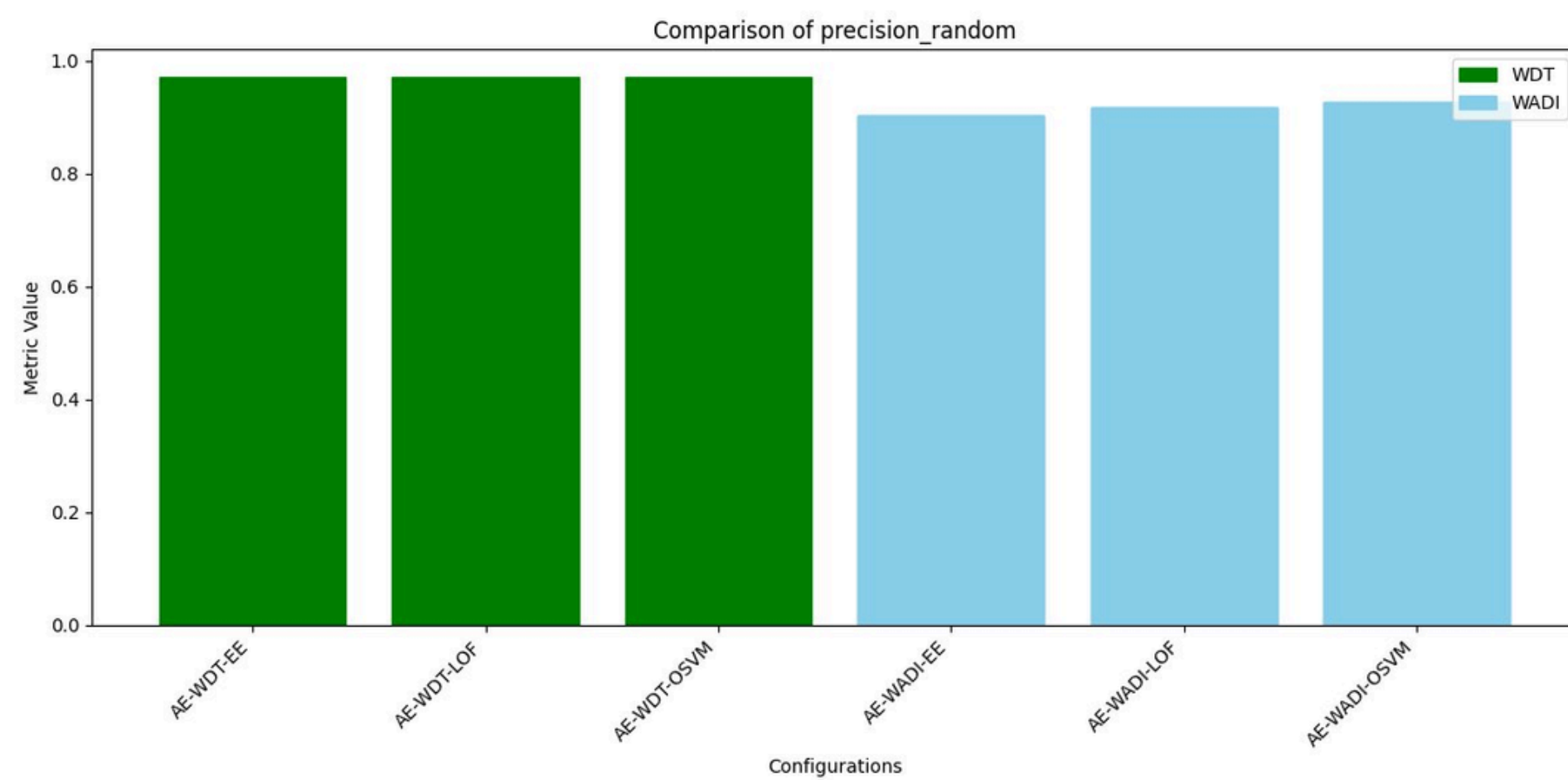
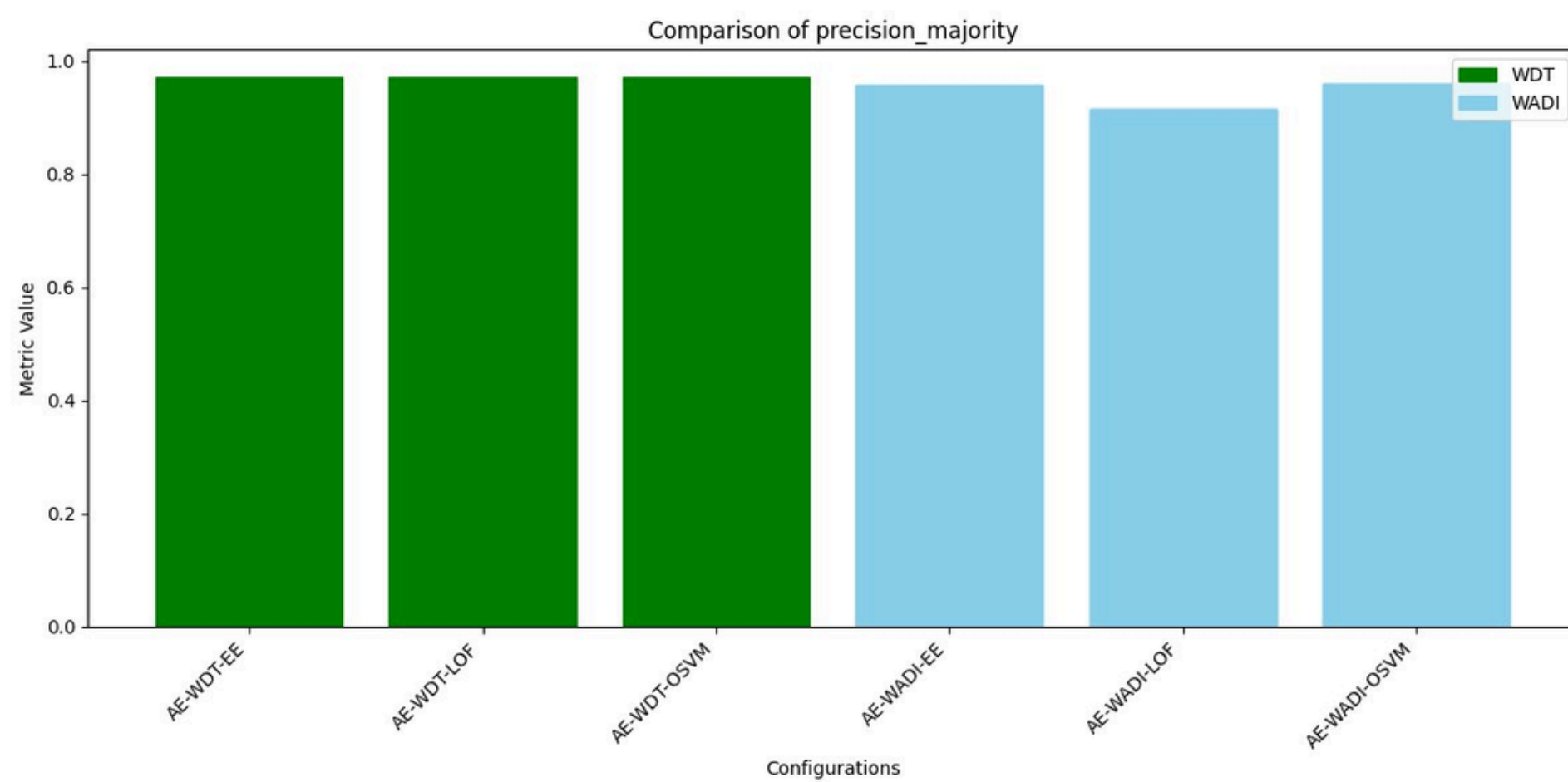


CUSUM EWMA



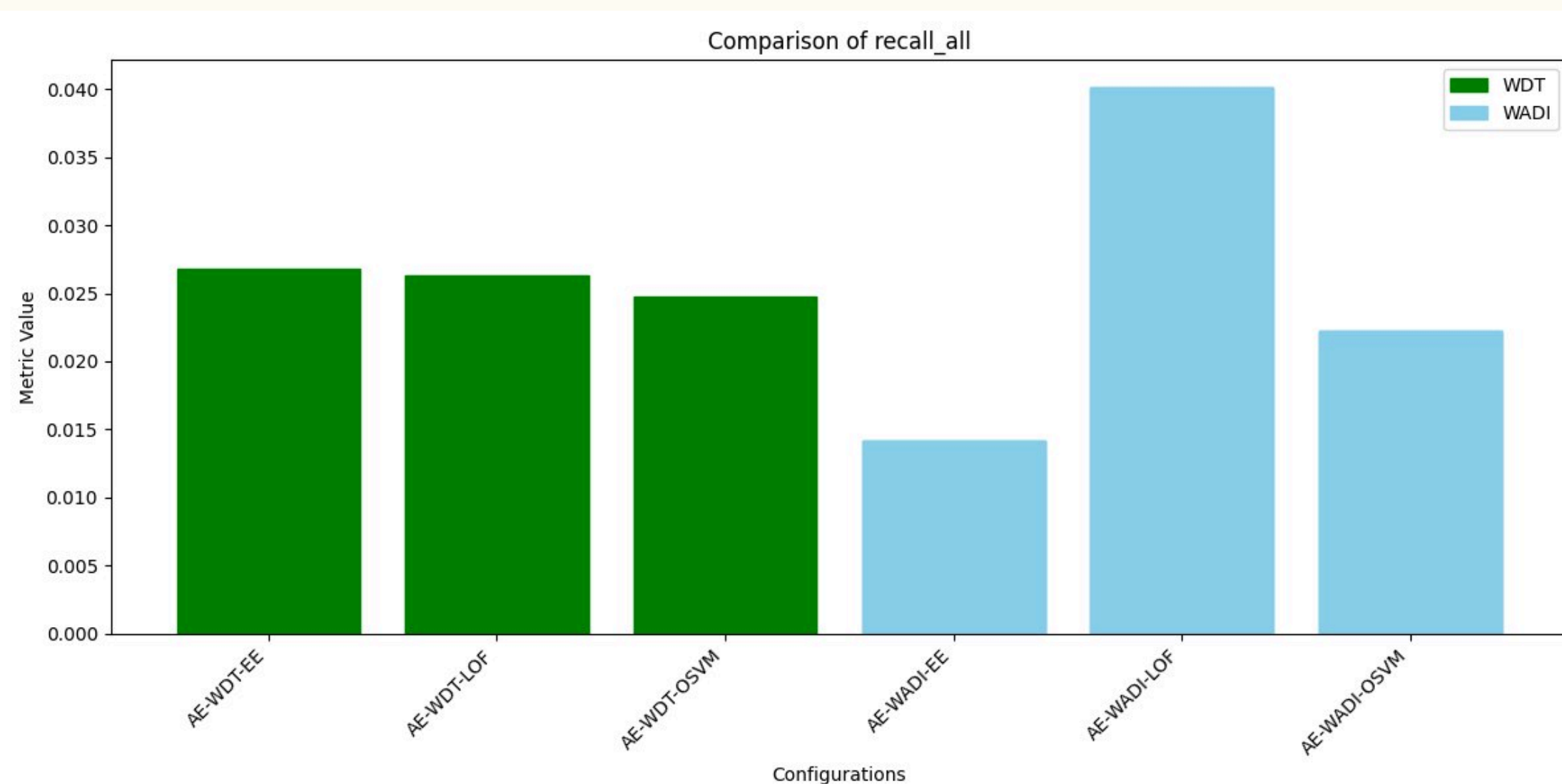
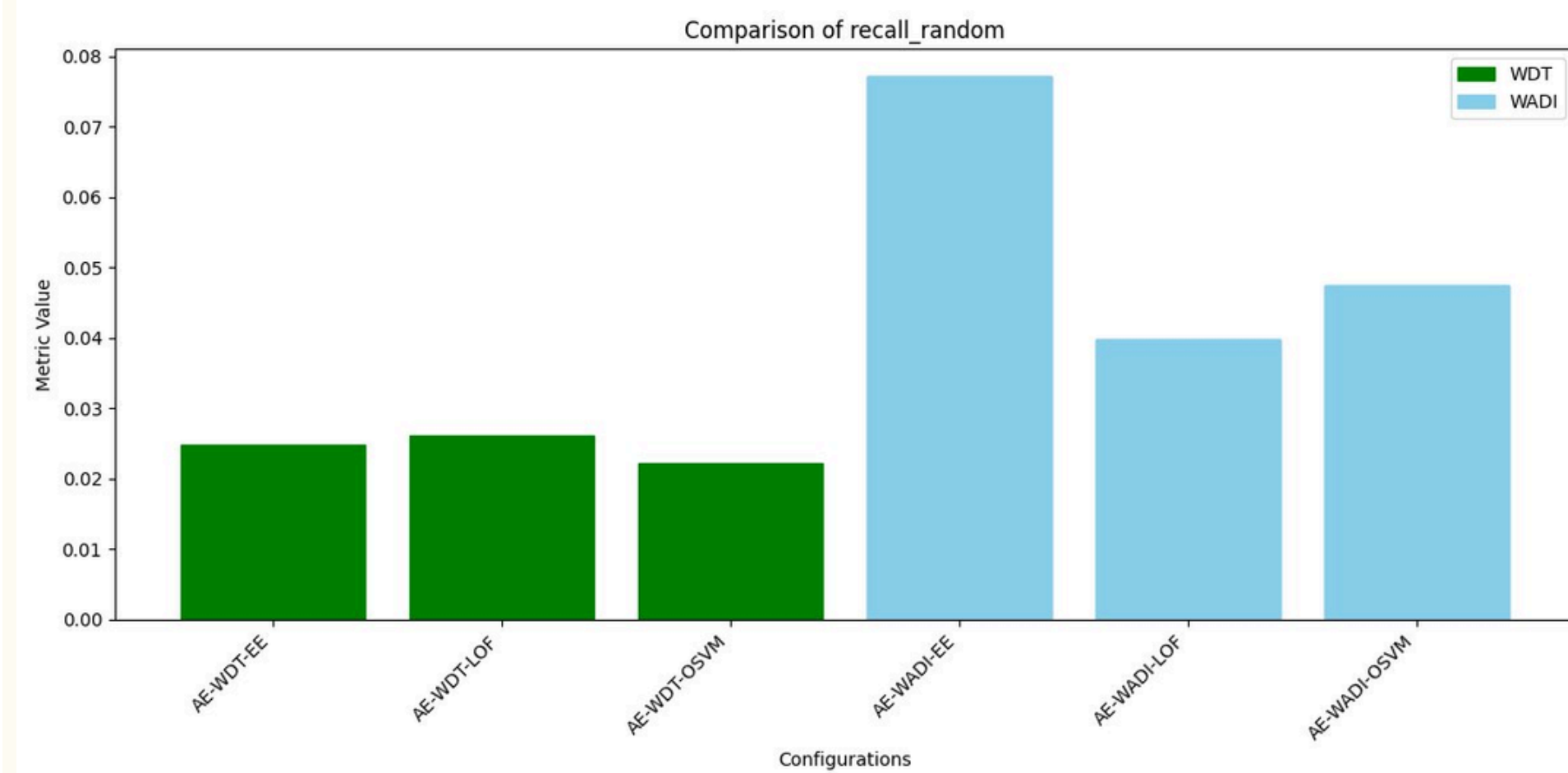
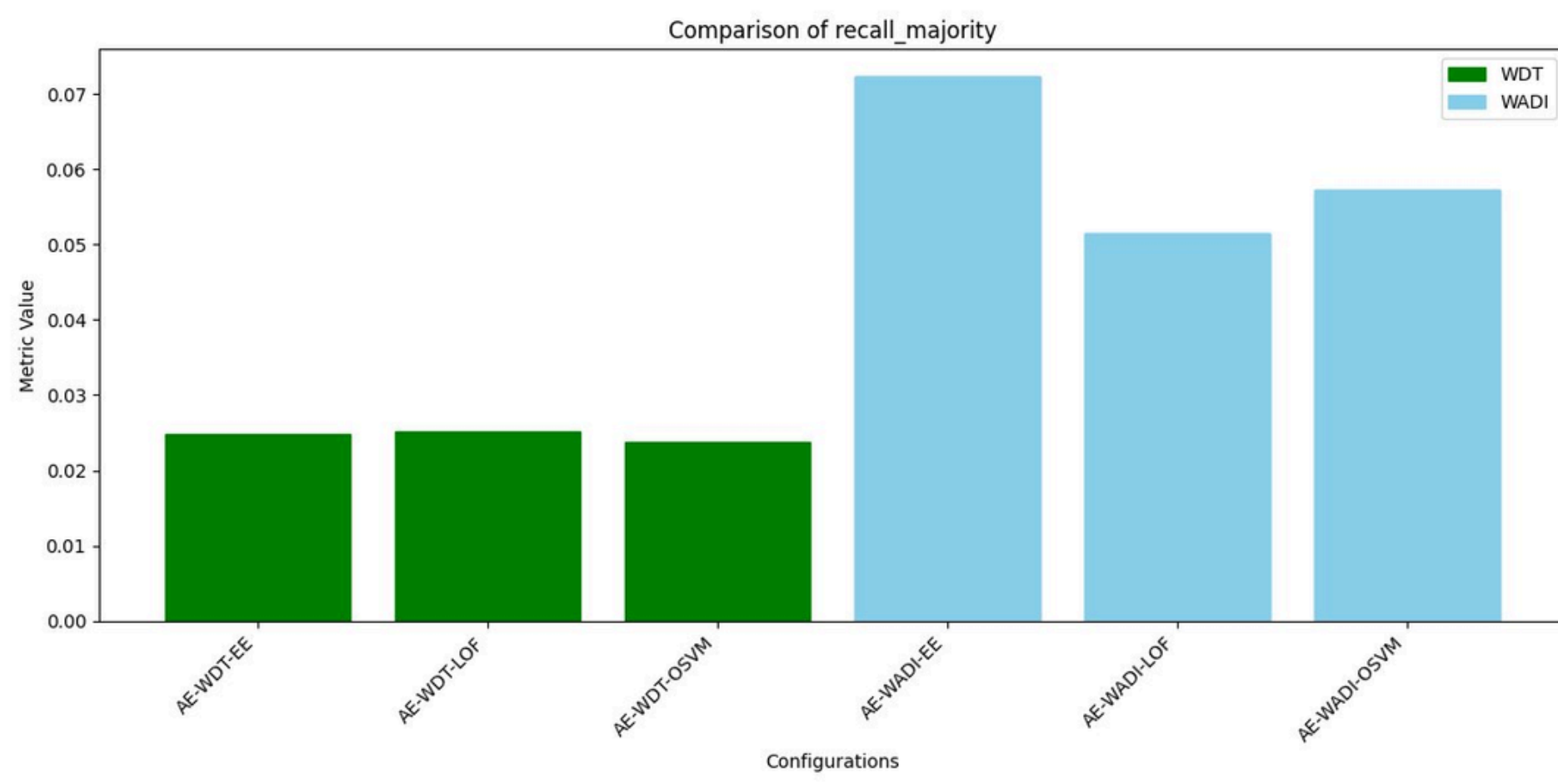
SCENARIO 1

- A line graph comparing precision, recall for different classification models.
- It helps evaluate the model's effectiveness in detecting true anomalies while minimizing false positives and false negatives.
- A higher precision indicates fewer false positives, while a higher recall ensures more anomalies are detected.



ENSEMBLE CLASSIFIER

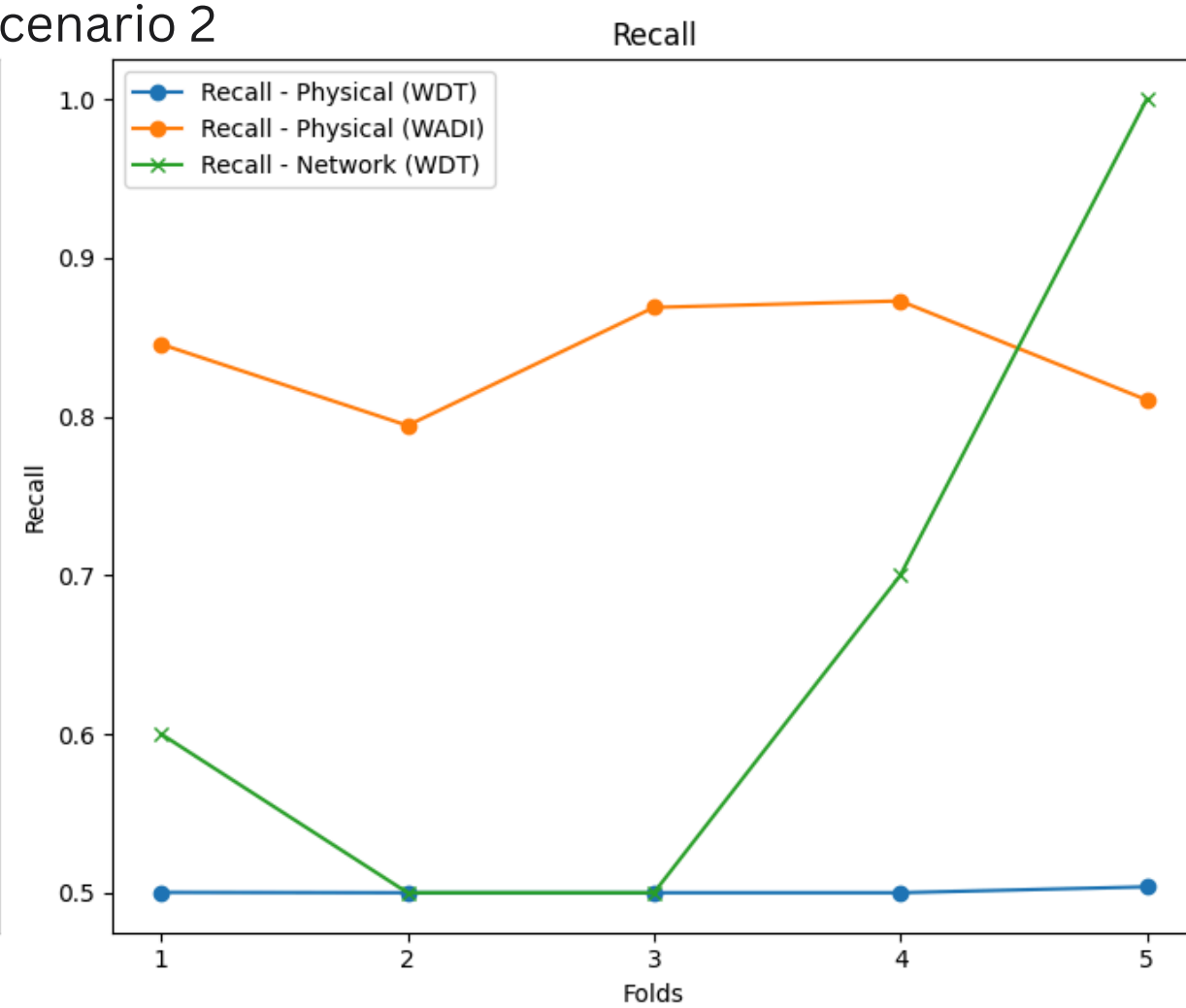
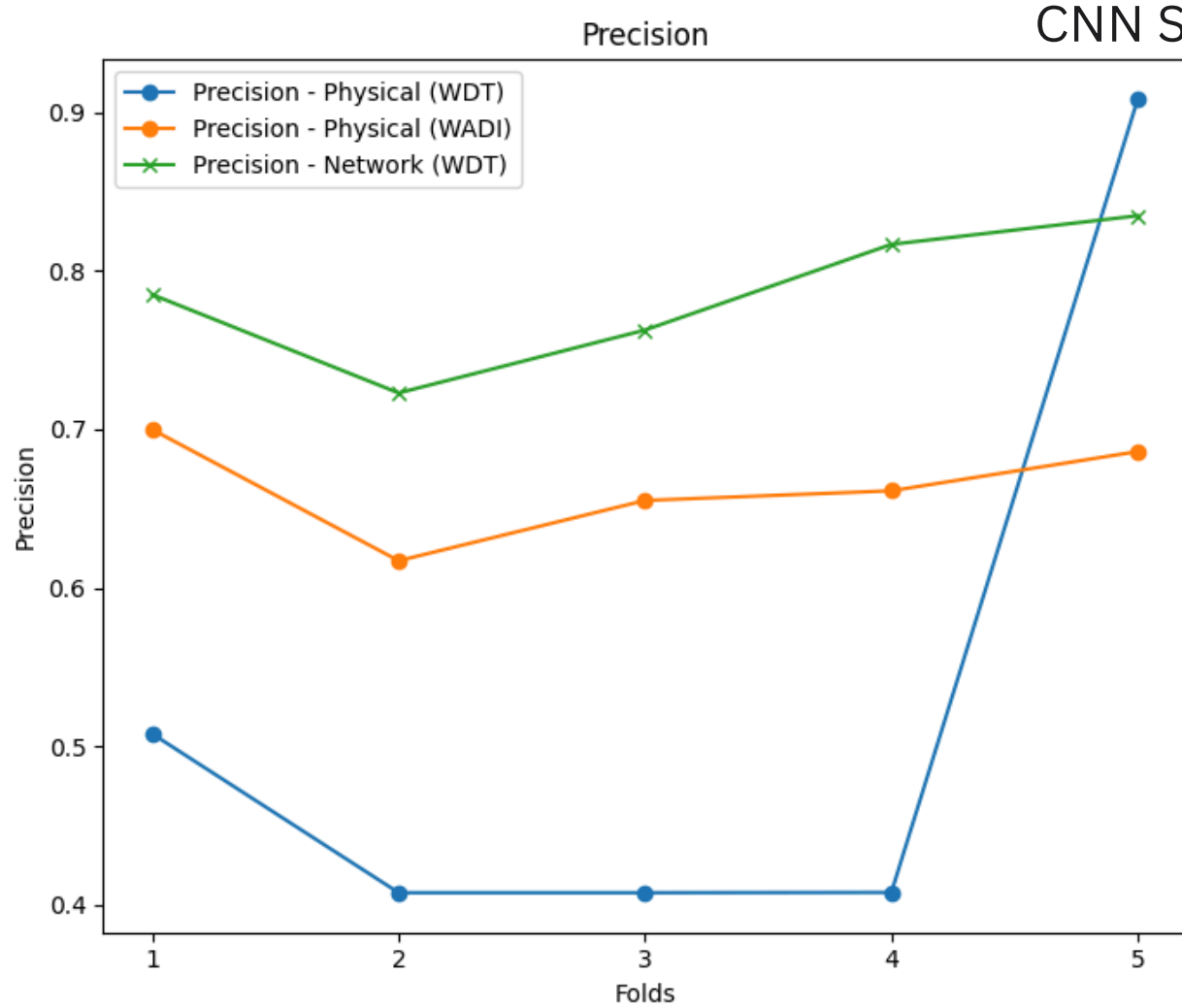
- A bar chart comparing precision for different classification models.
- It helps evaluate the model's effectiveness in detecting true anomalies while minimizing false positives and false negatives.
- A higher precision indicates fewer false positives, while a higher recall ensures more anomalies are detected.



ENSEMBLE CLASSIFIER

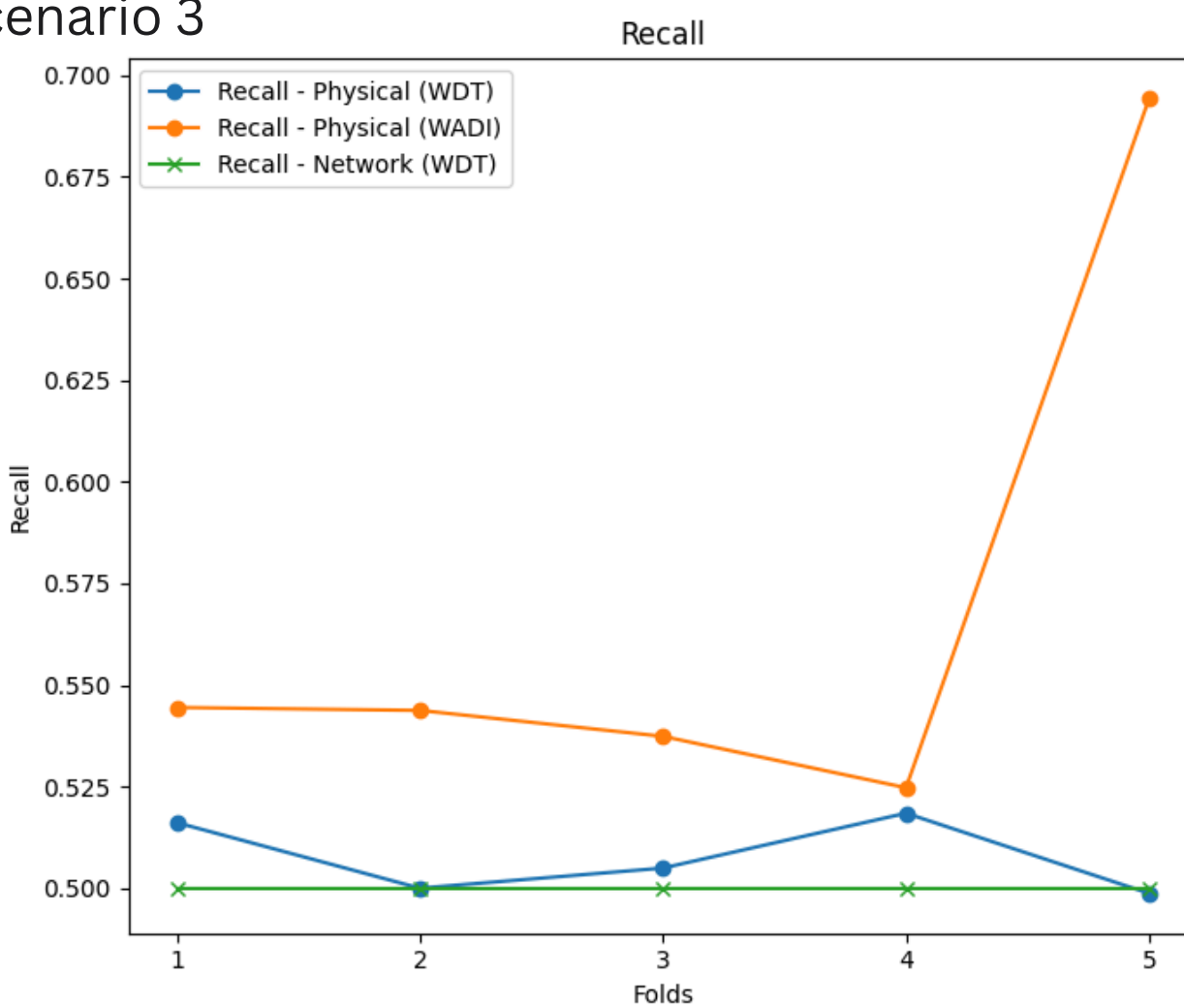
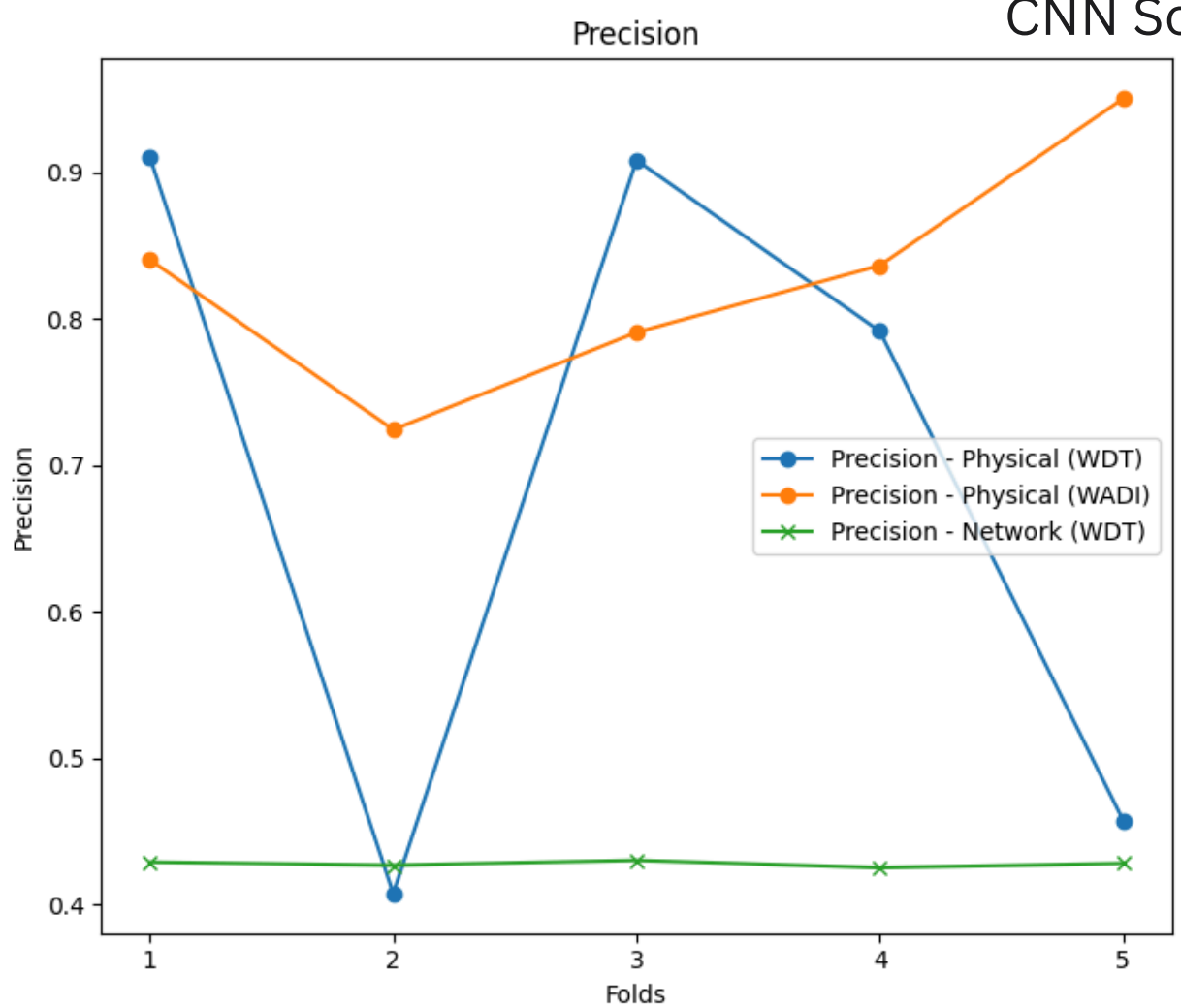
- A bar chart comparing recall for different classification models.
- It helps evaluate the model's effectiveness in detecting true anomalies while minimizing false positives and false negatives.
- A higher precision indicates fewer false positives, while a higher recall ensures more anomalies are detected.

CNN Scenario 2



- A line graph comparing precision, recall for different classification models.
- It helps evaluate the model's effectiveness in detecting true anomalies while minimizing false positives and false negatives. A higher precision indicates fewer false positives, while a higher recall ensures more anomalies are detected.

CNN Scenario 3



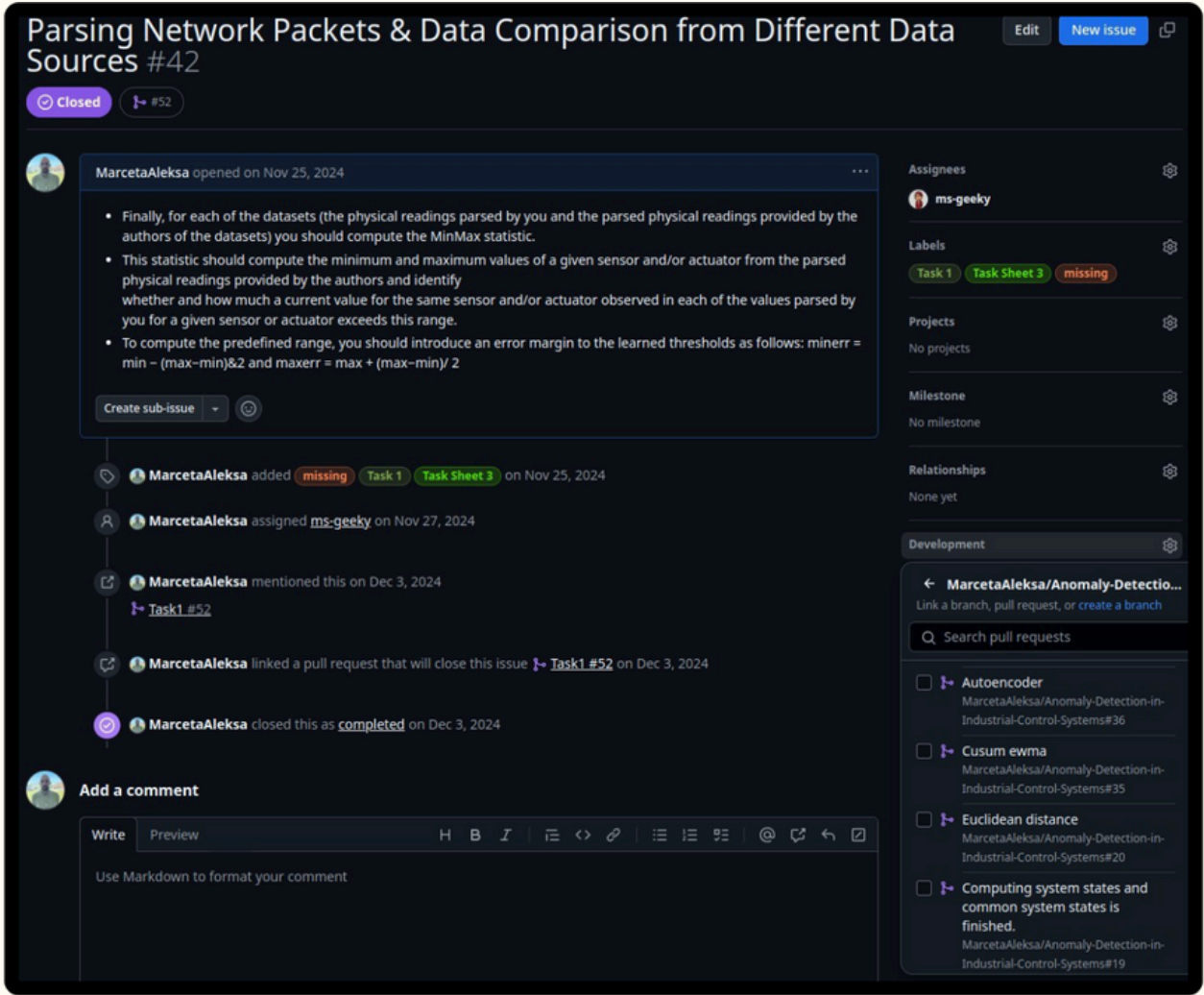
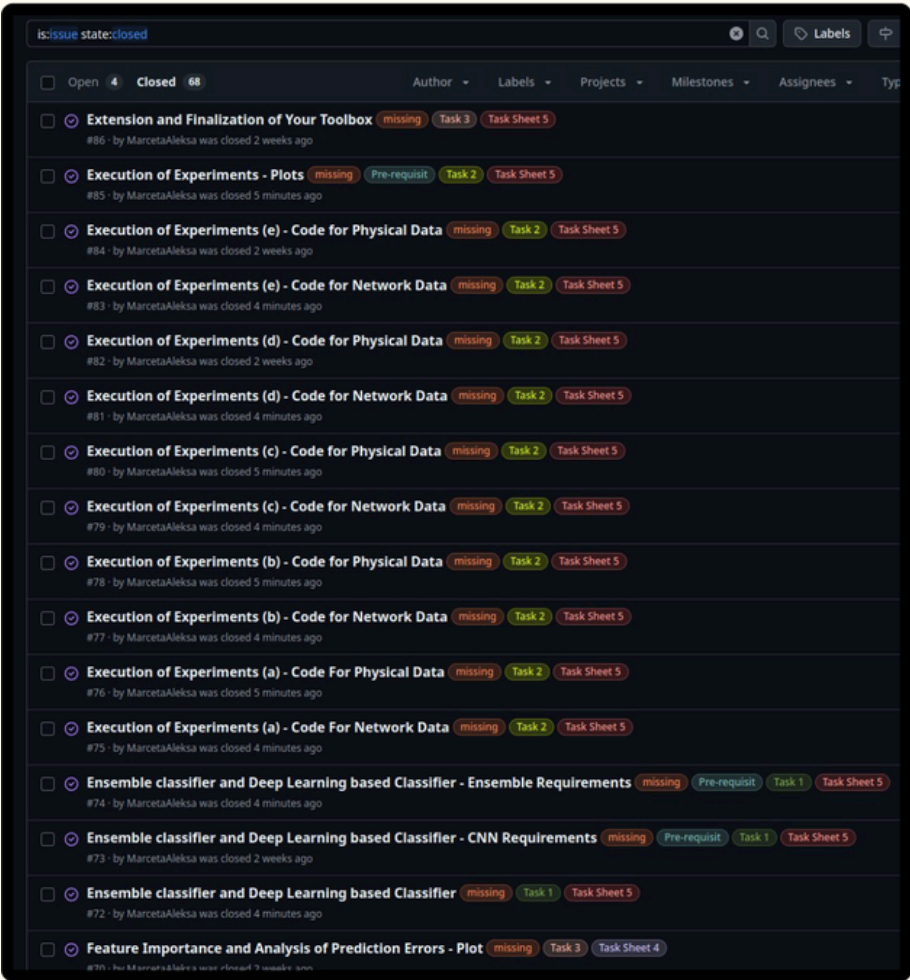
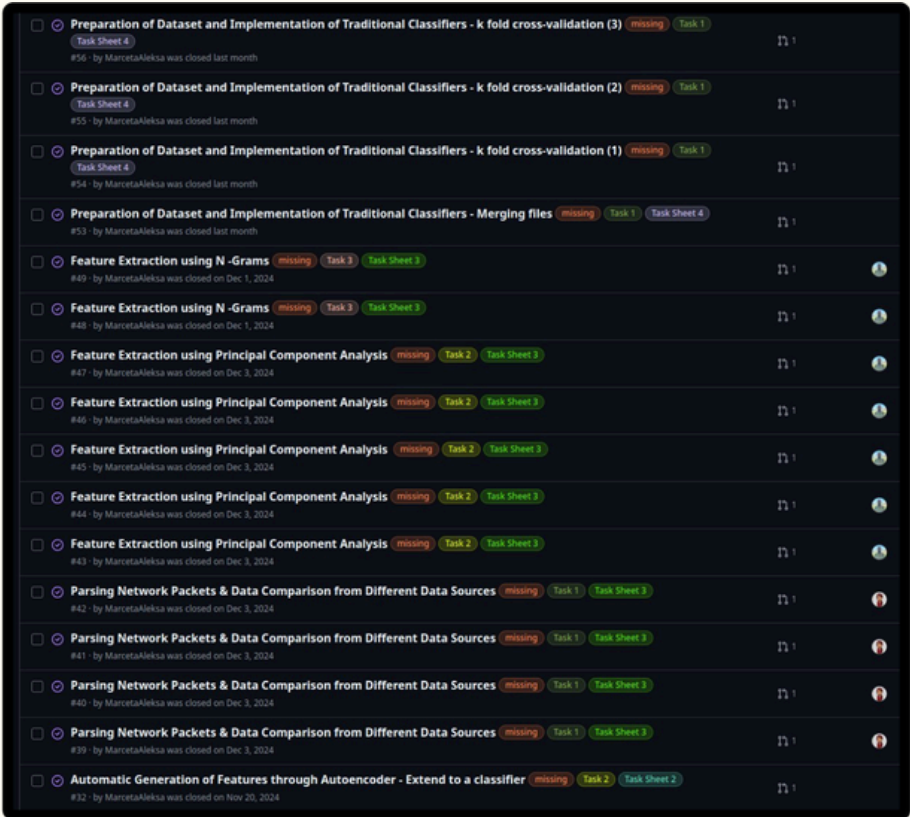
LESSONS LEARNT

- ◆ Statistical methods provided quick but less flexible detection mechanisms.
- ◆ Traditional Machine learning models offered a balance between efficiency and detection accuracy.
- ◆ Deep learning models excelled in high-accuracy detection but required significant computational power.
- ◆ Ensemble learning improved overall robustness by combining strengths of multiple classifiers.
- ◆ Feature selection (PCA, Autoencoder, N-Grams) played a crucial role in optimizing model performance.

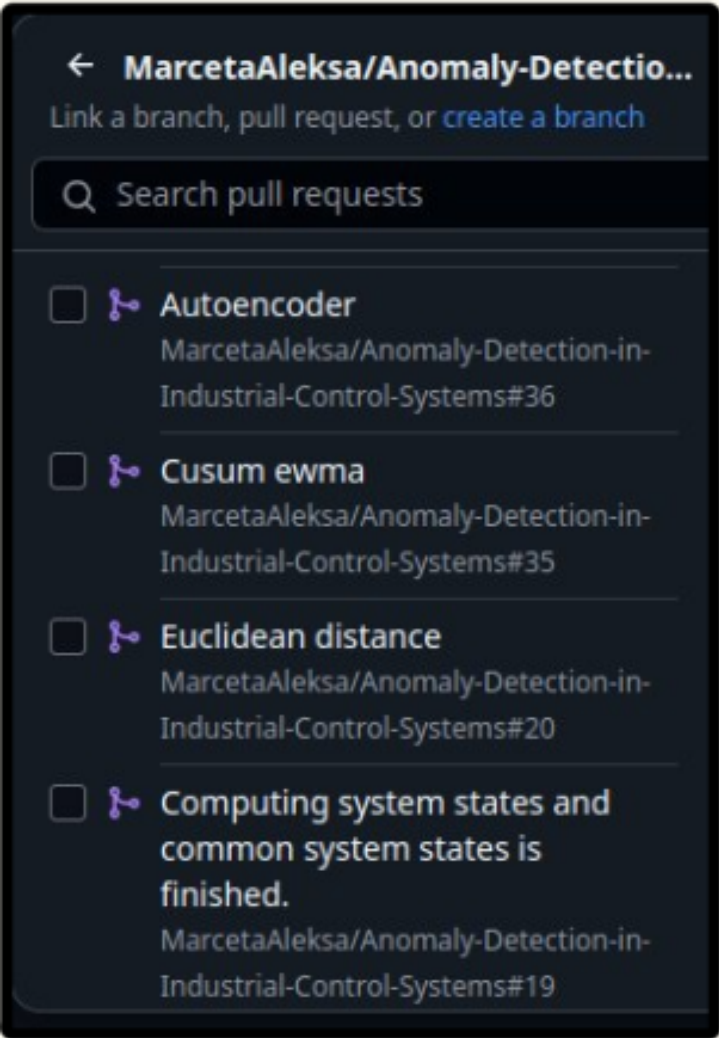
LIMITATIONS

- ◆ Time constraints towards the end of the project led us to be more stressed resulting in slightly bad results
- ◆ Size of wadi dataset - large -> execution of scenarios took too long
- ◆ Description of attacks in datasets limited our approach because we had to look for patterns in attacks
- ◆ New approach - we looked for sequence of attacks present and one sequence would reflect one attack type
- ◆ Repetition of readings in different folds - inaccurate predictions because not representative of real world data
- ◆ In very few cases, we used manual tuning instead of automatically tuning parameters.

GITHUB CODE ORGANIZATION



- 💡 Collaboration Made Easy
- 💡 Task Management & Issue Tracking



THANK YOU

ANY QUESTIONS?